

비전 Edge AI**무선 액추에이터 제어**

- STEP 00 시스템 개요
- STEP 01 모니터·출력 구성
- STEP 02 준비물
- STEP 03 분류 모델 만들기
- STEP 04 비전 노드 펌웨어
- STEP 05 코어측 모니터 UI
- STEP 06 제어 노드 (서보)
- STEP 07 독립 시연용 웹 출력
- STEP 08 게이지 다이얼 제작
- STEP 09 통합 테스트
- STEP 10 트러블슈팅·확장

공주고등학교

SW·AI 융합교육 · 2026-06

EDGE AI × BLE × ACTUATOR

비전 Edge AI 프로젝트 — 교사용 매뉴얼

카메라 보드(Nicla Vision)가 기기 안에서 스스로(온디바이스 추론) 물체를 판단합니다. 무거운 영상은 **USB 선 (유선)**으로 PC 화면에 실시간 전송(스트리밍)하고, 가벼운 판단 결과(단 3바이트)는 블루투스(BLE 무선)로 제어 보드(Nano ESP32)에 보내 서보 바늘과 OLED를 구동합니다 — 데이터의 무게에 맞게 길을 나눈(매체-부하 매칭) 엣지 AI 시스템입니다. 첫 응용 예제는 "추론 결과를 서보 바늘이 가리키는 비전 게이지"이며, 동일한 코어 위에서 분리수거 안내, 출입 감지 도어 제어 등으로 확장할 수 있습니다.

Arduino Nicla Vision

Arduino Nano ESP32

Edge Impulse

BLE GATT

USB 영상 스트림

SG90 Servo

Grove OLED

Streamlit

STEP 00

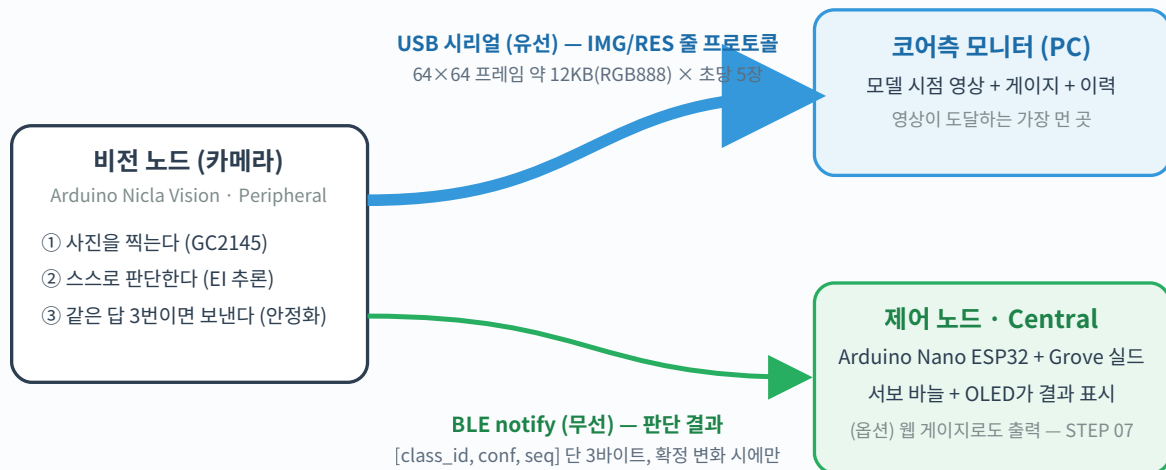
시스템 개요 — 영상은 유선으로, 판단은 무선으로

이 시스템의 설계 원칙은 "전송 매체를 데이터의 무게에 맞춘다"입니다. 용량이 큰(대역폭 부담이 큰) 영상은 USB 선(유선)으로 보내고, 몇 바이트면 충분한 판단 결과만 무선(BLE)으로 보냅니다. 덕분에 무선 구간은 끝까지 가볍고 안정적입니다.

먼저 정리하는 용어 12가지 (원어 병기)

학생용 매뉴얼과 같은 표현 체계를 쓰되, 수업 중 정확한 용어 안내를 위해 원어를 병기했습니다.

용어	쉬운 설명
엣지 AI (edge AI · 온디바이스 AI)	인터넷 서버로 보내지 않고, 기기 안에서 AI가 스스로 판단하는 방식.
추론 (inference)	학습을 마친 AI가 새 입력을 보고 답을 내는 일.
분류 모델 / 라벨 (classification / label)	정해진 보기 중 하나를 고르는 AI / 그 보기에 붙인 이름표.
펌웨어 / 스케치 (firmware / sketch)	보드 안에 넣어 두고 돌리는 프로그램 / 아두이노에서의 호칭.
업로드 (upload)	PC에서 작성한 프로그램을 USB로 보드에 옮겨 넣는 것.
BLE (Bluetooth Low Energy)	작은 데이터를 저전력으로 보내는 근거리 무선. 송신측이 Peripheral, 수신측이 Central.
바이트 (byte)	데이터 크기의 기본 단위. 한글 한 글자 2~3바이트, 사진 한 장 수만 바이트.
서보 모터 / 액추에이터 (servo / actuator)	신호를 받아 정해진 각도로 도는 모터 / 움직이는 장치의 총칭.
GND (ground · 공통 접지)	전기의 기준 0V 선. 장치들이 같은 기준을 공유해야 신호가 올바르게 읽힘.
안정화 필터 (stability filter)	같은 답이 3번 연속 나올 때만 확정하는 규칙 — 오인식 떨림 방지.
양자화 (quantization, int8)	모델 내부 수치를 작은 단위로 표현해 용량·연산을 줄이는 경량화 기법.
base64	사진 같은 이진 데이터를 글자들로 바꿔 한 줄 텍스트로 보내는 인코딩.



무거운 영상은 굵은 파란 길(유선)로, 가벼운 판단은 가는 초록 길(무선)로 — 길의 굵기가 곧 데이터의 무게입니다

왜 이런 구조인가?

- **길과 짐의 짝 맞추기 (매체-부하 매칭)** — 블루투스의 실제 전송 속도(실효 처리량, 초당 수~수십 KB)로는 영상을 보낼 수 없고, 그럴 필요도 없습니다. 영상은 유선으로, 결론은 무선으로 — 각 길에 잘 어울리는 짐만 맡깁니다.
- **역할 분리** — 비전 노드는 "보고 판단", 제어 노드는 "받아서 행동", PC는 "사람에게 보여주기". 실제 IoT의 표준 구조(센서 노드 ↔ 액추에이터 노드 ↔ 모니터링)를 그대로 체험합니다.
- **전송량의 통찰** — 무선으로는 영상이 아니라 **판단 결과 단 3바이트**만 이동합니다. "엣지에서 처리하고 결론만 보낸다"는 엣지 AI의 핵심 철학이 보내는 데이터(페이로드)의 크기에 그대로 드러납니다.

핵심 개념 ① — 3바이트 무선 프로토콜

BLE로 흐르는 데이터는 `[class_id, confidence, seq]` 단 3바이트입니다. `class_id`는 인식된 물체 번호(0~4), `confidence`는 확신도(0~100%), `seq`는 패킷 순번. 이 단순함이 BLE의 낮은 대역폭과 무선 공존 환경에서도 시스템을 안정적으로 만듭니다.

핵심 개념 ② — 줄 단위 시리얼 프로토콜

USB로 흐르는 데이터는 줄 단위 텍스트입니다: `IMG:<base64 프레임> 줄`과 `RES:{"cls":2,"conf":91} 줄`. 각 줄 앞에 `IMG:`, `RES:` 같은 머리말(접두어)을 붙여 구분하므로 PC 쪽에서 읽는 코드(파서)가 몇 줄이면 끝납니다. `base64`는 사진(이진 데이터)을 글자들로 바꿔 한 줄로 보내는 인코딩입니다. 영상은 모델 입력 그대로인 64×64 — "AI는 이 작은 그림으로 판단한다"를 보여주는 것 자체가 교육 포인트입니다.

핵심 개념 ③ — 두 보드, 두 BLE 라이브러리, 하나의 규격

Nicla Vision(mbed 코어)은 [ArduinoBLE](#) , Nano ESP32(esp32 코어)는 [BLEDevice.h](#) 를 씁니다. 코드 문법은 전혀 다르지만 무선 규격(BLE GATT)이 국제 표준이라 서로 완벽히 통신합니다. "라이브러리는 방언, 전파는 표준어" — 이질적 조합이 동작하는 것이 표준의 힘을 보여줍니다.

STEP 01

모니터·출력 구성 — 기본 + 옵션

사람이 결과를 보는 경로는 **기본 구성(코어측 PC 모니터)** 하나와, PC 없이 시연할 때 추가하는 **독립 시연 옵션** 둘로 정리됩니다. 서보(물리 게이지)와 OLED(현장 디지털 표시)는 제어 노드에 직결돼 있어 **어느 구성에서든 항상 동작합니다.**

기본

코어측 PC 모니터 — 영상 + 게이지 + 이력 ★ 개발·수업의 중심

비전 노드의 USB 시리얼을 PC가 직접 읽어 Streamlit으로 표시 — 유일하게 라이브 영상이 보이는 경로

- 모델이 보는 64×64 영상을 실시간 표시 (약 5fps — 추론 주기와 동기)
- 인식 결과 게이지 + 확신도 + 시간별 이력 로그까지 **가장 풍부한 UI**
- 무선 부담 0 추가 — 영상은 유선이므로 BLE는 그대로 한가함
- 켜고 끄기 = USB 연결 여부. 안 켜면 시스템은 무선 액추에이터만으로 자율 동작

옵션 ①

SoftAP 독립 웹 게이지

PC 없는 전시·이동 시연용 — 제어 노드가 스스로 Wi-Fi 핫스팟이 되어 게이지 웹페이지 서빙

- 폰 Wi-Fi에서 "VisionGauge" 접속 → [192.168.4.1](#) — 외부 인프라 0, 어디서든 재컴파일 없이 동작
- 영상은 없음 (영상은 코어측 전용) — 게이지·라벨·확신도 표시
- 접속한 폰은 그동안 인터넷 단절, 동시 접속 ~4대 권장

옵션 ②

공유기 경유 웹 게이지 (Station)

교실 전체가 각자 폰으로 동시에 보는 시나리오용 — 기존 Wi-Fi망에 합류

- 접속 수 제한 사실상 없음, 시청자 폰의 인터넷 유지
- 환경마다 SSID/PW 재컴파일 필요 + 학교망 AP isolation 사전 확인 필수

한눈에 비교

평가 축	기본 코어측 모니터	옵션① SoftAP 웹	옵션② 공유기 웹
라이브 영상	◎ 64×64 ~5fps	×	×
UI 표현력	◎ Streamlit	○ 단일 페이지	○ 단일 페이지
외부 인프라	PC	없음	공유기+망정보
ESP32 무선 부담	◎ BLE 단독	○ BLE+WiFi	○ BLE+WiFi
이동·설치 간편성	○ PC 동반	◎ 전원만	△ 망 재설정
동시 시청자 수	○ PC 화면 공유	○ ~4대	◎ 제한 없음
환경별 재컴파일	불필요	불필요	필요(SSID/PW)

운용 가이드

- ① 개발·수업: 기본 구성만으로 진행 — 영상으로 모델의 시점을 확인하며 데이터 보강·임계값 튜닝까지.
- ② 전시·이동 시연: USB를 뽑고 옵션①을 겁니다 — 보드 2개 + 폰으로 완전 독립.
- ③ 교실 단체 시청: 옵션② — 단, AP isolation 확인 후.

기본 구성과 옵션은 배타적이 아닙니다. 옵션 코드를 넣어둔 채 USB만 꽂았다 뺐었다 하며 상황에 맞게 운용하세요.

STEP 02

준비
물

구분	품목	비고
비전 노드	Arduino Nicla Vision	GC2145 2MP 카메라·Wi-Fi/BLE 내장
제어 노드	Arduino Nano ESP32 + Grove Shield for Arduino Nano	⚠ 실드 전원 스위치 3V3 고정 (5V 금지)
액추에이터	SG90 마이크로 서보	180° 회전형 (360° 연속회전형 아님!)
표시 모듈	Grove OLED Display 1.12"	V2=SSD1327(0x3E) / V3=SH1107(0x3C) — I2C 스캐너로 확인
서보 전원	LM2596 FND 전압표시 강압 컨버터 + 9~12V 어댑터	⚠ 무부하 5.0V 설정 후 연결 — STEP 06
배선 소품	Grove-to-male 점퍼 케이블, Grove 4핀 케이블, 헤더핀(모듈 납땜용), 소형 일자 드라이버	드라이버는 트리머(W103) 조정용
공작 재료	두꺼운 종이/포맥스, 서보 혼, 양면테이프	게이지 다이얼판 제작용
소프트웨어	Arduino IDE 2.x + esp32 보드 패키지 + Arduino Mbed OS Nicla Boards	보드 매니저에서 각각 설치
라이브러리	ESP32Servo, U8g2 (Nano용) · ArduinoBLE (Nicla용)	라이브러리 매니저
클라우드 계정	Edge Impulse (무료)	Nicla Vision 공식 지원 보드
코어측 모니터 (기본)	PC + Python (pyserial, streamlit, plotly, numpy)	pip install pyserial streamlit plotly numpy
옵션① 추가	없음	폰/태블릿 1대면 충분
옵션② 추가	공유기 SSID/비밀번호, AP isolation 해제 확인	학교망은 관리자 확인 권장

인식 대상 정하기

모두이 자유롭게 선택 — 교실에서 구하기 쉽고 모양이 서로 뚜렷이 다른 물체 **2~5종**을 고르세요. 예: 물병·가위·마커펜·컵·안경·키보드·책·연필꽃이 등 무엇이든 좋습니다. 모양이 비슷한 물체(펜 vs 연필, 휴대폰 vs 지갑)는 피하는 것이 정확도에 유리합니다. unknown 클래스는 따로 학습시키지 않아도 됩니다 — 펌웨어의 임계값 필터(0.70 미만 → "모름")가 자동으로 처리합니다.

라벨 이름은 짧은 영문 단어로 지으세요 (예: `cup`, `pen`, `book`, `bottle`, `marker`). OLED 화면에 표시되므로 8자 이내가 보기 좋습니다. 길어도 화면에서 자동으로 글씨가 줄어들지만, 짧을수록 크고 선명하게 보입니다.

STEP 03

분류 모델 만들기

담당: Edge Impulse Studio (클라우드) — 학습 연산은 보드가 아니라 클라우드에서 일어납니다

WHAT — 이 단계가 하는 일

물체 사진 수백 장을 모아 클라우드에 올리고 라벨을 붙여 신경망을 학습시킵니다. 완성된 모델을 Nicla Vision용 Arduino 라이브러리 형태로 내려받습니다.

WHY — 왜 이렇게 하는가

학습에는 GPU급 연산이 필요해 보드에서는 불가능합니다. "학습은 클라우드, 추론은 엣지"로 나누면, 우리가 받는 건 학습이 끝난 모델의 추론 코드뿐이라 작은 보드에서도 돌아갑니다.

3-0. 사전 준비 — Nicla Vision에 Edge Impulse 펌웨어 굽기

공장 출고 상태의 Nicla Vision은 EI Studio와 직접 통신하지 못합니다. WebUSB·CLI 데몬·데이터 수집 모드는 모두 **EI 전용 펌웨어가 올라간 상태**에서만 동작합니다. 학생 차시 진입 전에 모든 모듈 보드에 미리 구워 두는 것을 권장합니다(모듈당 한 번만).

1. EI 펌웨어 zip 받기 — docs.edgeimpulse.com/docs/edge-ai-hardware/mcu/arduino-nicla-vision

"Update the firmware" 섹션. 압축 풀면 `firmware.bin` + OS별 flash 스크립트(`flash_windows.bat` / `flash_mac.command` / `flash_linux.sh`).

2. **arduino-cli** 설치 — flash 스크립트가 내부적으로 호출합니다. PowerShell 한 줄:

```
winget install ArduinoSA.CLI
```

설치 후 모든 명령 창을 닫고 새로 열기(PATH 갱신은 새 세션에만 적용). 검증: `arduino-cli version`. 학교망에서 winget이 막혔다면 zip 다운로드 → 임의 경로에 두고 그 경로를 PATH에 수동 추가.

3. **부트로더(DFU) 진입** — Nicla Vision 리셋 버튼을 빠르게 더블 클릭. 성공 신호는 녹색 LED 호흡(약 1Hz). 동시에 Windows 장치 관리자에서 포트 이름이 변경되거나 새 COM이 잡힙니다.
4. **플래시 실행** — 호흡 LED 확인 후 5초 안에 `flash_windows.bat`. 첫 실행 시 Mbed Nicla Boards 코어를 약 200MB 자동 설치합니다(이후 캐시됨). "Flashed your Arduino Nicla Vision" 출력 시 완료.
5. **검증** — EI Studio Data acquisition → "Connect a device". CLI 직접 사용 시 `edge-impulse-daemon`으로 데몬 기동 후 같은 화면에서 연결.

학생 차시에서 빈발하는 실패 패턴

- ① **arduino-cli not found** — 설치 후 cmd 창을 새로 안 연 케이스(가장 흔함).
- ② **No connected board found** — 더블탭 타이밍 실패. 호흡 LED 확인 전에는 실행 금지로 안내.
- ③ **flash bat 더블클릭 시 창이 0.5초 만에 닫힘** — 에러 메시지를 보려면 cmd에서 직접 실행해야 한다고 안내.
- ④ **"Failed to connect to device" (Studio 5초 타임아웃)** — 펌웨어 미적용 상태에서 Studio에 바로 접속 시도.
- ⑤ **부트로더 모드에서 보드 인식 실패** — Arduino IDE 보드 매니저에서 "Arduino Mbed OS Nicla Boards"를 한 번 설치하면 드라이버가 함께 들어옵니다.
- ⑥ **시리얼 포트 점유** — Arduino IDE 시리얼 모니터, PuTTY, monitor.py 등이 같은 COM을 잡고 있으면 데몬이 못 엽니다.

교실 운영 팁

펌웨어 굽기를 학생에게 맡기면 차시당 30분이 사라집니다. **방과 후 한 번에 전체 모뎀 보드를 일괄 작업**해 두시는 것을 권장 — USB 허브 + 보드 모두 부트로더 진입 후 같은 PC에서 순차 플래시. 또는 한 보드에 펌웨어를 굽고 나머지는 학생이 따라 하게 하되, 위 ①~⑥ 트러블슈팅 카드를 모뎀에 배포하면 자체 해결률이 크게 올라갑니다.

3-1. 프로젝트 생성과 데이터 수집

1. edgeimpulse.com 로그인 → **Create new project** (예: `vision-actuator`)
2. **Data acquisition** 탭 → 우측 USB 아이콘 → "Connect a device" → Nicla Vision이 WebUSB로 잡힘 (3-0 EI 펌웨어가 굽혀 있어야 인식됨)
3. **Sensor: Camera (64×64)** 선택. 96 이상은 한 프레임이 너무 커서 USB 버퍼·메모리 단편화가 누적되어 끊김이 잦습니다.

4. 각 클래스마다 **Label** 칸을 먼저 설정 → 물체 배치 → **Start sampling** (Label과 Sample name은 별개 — 학생들이 자주 헷갈리는 지점)
5. 클래스당 **60~80장**, 각도·거리·배경·조명을 다양하게.
6. 업로드 시 학습용 80% / 테스트용 20% 자동 분할 확인.

데이터 품질이 모델의 전부

실제 시연할 거리·배경·조명과 비슷하게 촬영. "책상 위 30cm"에서 시연할 거면 학습 사진도 그렇게. 학습-시연 환경의 도메인 차이가 정확도 하락의 가장 흔한 원인입니다.

WebUSB 수집 안정성 — 학생 차시 운영 가이드

WebUSB는 EI Studio가 보드와 직접 통신하는 신생 방식이라 학생 PC 환경에 따라 끊김이 잦습니다. CLI([edge-impulse-daemon](#))가 더 안정적이지만 Node.js + npm 설치가 필요해 학생 차시엔 부담이 큼니다. WebUSB로 가되 아래를 지키게 하세요:

- ① **해상도 64×64 고정** — 96은 USB 버퍼·메모리 단편화로 끊김 빈발. 학생 차시 진입 전 안내 필수.
- ② **한 번에 길게 찍지 않기** — 클래스당 60장을 20×3회로 끊기. 사이사이 보드가 회복.
- ③ **Chrome + 다른 탭 닫기** — WebUSB는 브라우저 CPU 점유에 민감.
- ④ **끊김 응급처치 카드**를 모둠에 배포: 새로고침 → 재연결 → USB 재삽입 → 보드 리셋. 30초 내 회복 가능.
- ⑤ **폴백 — 폰 업로드**: 특정 모둠 보드가 계속 불안정하면 Studio Data acquisition 화면의 "Phone" 아이콘 → QR 스캔 → 학생 폰 카메라로 촬영. 단 도메인 차이(폰 1080p vs Nicla 64) 발생하므로 Nicla 사진과 섞어 학습할 것.

3-2. 임펄스 설계와 학습

1. **Create impulse**: 좌측 Image data 박스 — Image width/height는 데이터 수집 해상도와 일치시키기(학생 차시 권장: **64×64**. 96 이상은 WebUSB 데이터 수집 시 끊김 빈발). Resize mode는 **Fit shortest axis**(기본값) 유지 — 카메라가 4:3으로 들어오므로 중앙 크롭이 자연스럽습니다.
2. **Add a processing block** → **Image** 선택. Color depth는 RGB 유지.
3. **Add a learning block** → **Transfer Learning (Images)**. ⚠ Classification(일반)이나 FOMO를 같이 추가하면 Output features가 6개로 잡혀 배포 시 어느 모델을 쓸지 매번 묻게 됩니다. **Transfer Learning** 단독이 정답.
4. 오른쪽 초록색 Output features가 클래스 수와 같으면 정상 → **Save Impulse**
5. 좌측 **Image** → Save parameters → **Generate features**. Feature explorer 산점도가 클래스별로 잘 갈리는지 확인 — 한 덩어리로 섞이면 학습이 의미 없으니 데이터 보강이 먼저입니다.
6. 좌측 **Transfer learning** → 기본 모델 유지, Training cycles 20, Learning rate 0.0005, Training processor **GPU** (EI 클라우드 자원이라 학교 PC 사양 무관) → **Save & train**. 학습 중 브라우저 닫아도 진행됩니다.

7. **Confusion matrix** 합격 기준: 85% 이상. 특정 클래스만 0%이면 그 클래스 데이터 부족 + 다른 클래스와 시각적으로 너무 비슷한 경우.
8. (권장) **Live classification**에서 실제 시연 환경의 빈 책상·무관한 물체로 확신도 분포 확인. 모든 클래스가 50% 미만으로 흩어지면 unknown 클래스 없이 임계값 필터로 충분합니다.

unknown 클래스 전략 — 둘 중 택일

- A. 학습에 unknown 클래스를 두는 경우:** 빈 책상·손·무관한 물체 5~6종을 섞어 60장 이상. 이 데이터의 "다양성"이 unknown 정확도의 전부 — 한 가지만 60장 찍으면 unknown만 0% 나옵니다.
- B. unknown 없이 임계값 필터만 쓰는 경우:** 두 진짜 클래스로만 학습하고, 펌웨어에서 `if (maxConf < 0.70)` unknown 처리. 모델이 가볍고 진짜 클래스 정확도가 더 잘 나옵니다. **학생 차시에는 B가 더 안정적입니다.**

학생 차시 빈번한 함정 — 라벨링

Data acquisition 화면의 **Label**(학습용 정답)과 **Sample name**(파일명)은 다른 항목입니다. 학생들이 흔히 Label을 한 번 설정해 두고 다른 물체를 찍어 라벨 불일치를 만듭니다. 매 클래스 변경 시 Label 칸부터 바꾸도록 안내. 잘못된 라벨은 클릭 → Edit label로 수정 가능.

3-3. 배포 (모델 내려받기)

1. **Deployment** 탭 → Deployment target: **Arduino library**
2. Inference engine: **EON™ Compiler** 기본 선택 유지 — 같은 모델을 더 작게 빌드(RAM 약 19%, 플래시 약 21% 절감). Nicla 메모리 여유가 빠듯하므로 끄지 마세요.
3. Model optimizations: **Quantized (int8)** 선택. 추론 속도 약 4배 향상(예: float32 116ms → int8 32ms), 정확도 손실 거의 없음.
4. **Build** → 1~2분 후 zip 자동 다운로드
5. Arduino IDE에서 **스케치** → 라이브러리 포함 → **.ZIP 라이브러리** 추가로 설치
6. **파일** → **예제** → **ei-(프로젝트명)-arduino** → **nicla_vision** → **nicla_vision_camera**가 보이면 성공

EON Compiler 의 정체

EI 자체 도구. 일반적 임베디드 흐름에서는 TFLite Micro라는 범용 인터프리터를 함께 올려 모든 연산자 코드를 보드에 실지만, EON Compiler는 학습된 모델이 실제 쓰는 연산만 추출해 전용 C++ 코드로 변환합니다. "전국 어디든 가는 SUV → 학교만 다니는 자전거"로 비유 가능. 인터프리터 제거로 메모리만 줄고 정확도는 그대로.

추론 시간이 예상치의 4배라면 — int8 빌드 확인

시리얼 모니터에서 추론 시간이 130~140ms로 나오면 십중팔구 **Unoptimized(float32)**로 빌드된 상태입니다. Studio Deployment에서 좌측 **Quantized (int8)**에 "Selected ✓" 배지가 보이는지 확인 후 재 빌드. EON Compiler까지 켜진 정상 빌드는 약 32ms 수준입니다(Nicla Cortex-M7 480MHz 기준).

STEP 04

비전 노드 펌웨어 — 단계별로 만들고 검증하기

담당: Arduino Nicla Vision — 캡처 → 추론 → 안정화 → [USB IMG·RES / BLE 3바이트] 출력. 한 번에 다 짓지 말고 레이어를 단계별로 추가.

WHAT — 이 단계가 하는 일

① 코어 추론 → ② USB 스트리밍 + 모니터 UI → ③ BLE 송신을 단계별로 레이어를 추가합니다. 매 단계 끝마다 눈으로 동작을 확인하므로 문제 격리가 쉽습니다.

WHY — 통합 빌드를 피하는 이유

BLE+카메라+USB 스트리밍이 동시에 안 되면 어디서 막혔는지 분리되지 않습니다. ArduinoBLE의 `BLE.poll()` 누락, 시리얼 가드 누락, EI 라이브러리 미설치 — 세 원인이 같은 증상(보드 멈춤)으로 수렴해 학생 차시에서 30분이 사라집니다.

4-1. 보드 설정 — 한 번만

- 보드 매니저: **Arduino Mbed OS Nicla Boards** 설치 → 보드 **Arduino Nicla Vision** 선택
- 라이브러리: STEP 03에서 받은 **EI zip**을 **.ZIP 라이브러리 추가**로 설치 (3-A에서 구운 펌웨어와는 별개 — 학습된 모델이 담긴 라이브러리입니다)
- (4-C에서 사용) **ArduinoBLE** 라이브러리 매니저로 설치. 4-A/4-B 단계에서는 필요 없습니다.
- 업로드 실패/포트 사라짐 시: **리셋 빠르게 2회** → 부트로더(녹색 LED 호흡) → 재업로드

4-A. 코어 테스트 — 추론 + 안정화 필터 (USB·BLE 없음)

가장 단순한 형태로 시작합니다. 카메라 → 추론 → 시리얼 모니터에 텍스트만. 학생 차시에서도 동일하게 이 단계부터 시작시키세요 — 이 단계가 통과하지 않으면 그 위 어떤 층도 의미가 없습니다.

1. 아래 코드를 새 스케치(`vision_node_step1_test.ino`)에 붙여넣고 업로드 (컴파일 1~2분). 학생 차시 진입 전에 교사가 한 번 직접 빌드해 보길 권장 — 학생 PC 환경에서 같은 컴파일 에러가 재현되는지 점검할 수 있습니다.

```

/* =====
 * Vision Gauge — STEP 4-A · 코어 테스트 펌웨어
 *
 * 이 파일에서 검증하는 것:
 * 1) 카메라(GC2145)가 정상 캡처되는가
 * 2) Edge Impulse 모델이 보드에서 추론을 돌리는가
 * 3) 안정화 필터(같은 답 3회 + conf >= 0.70)가 의도대로 작동하는가
 *
 * 이 파일에 없는 것 (다음 단계에서 추가):
 * - USB 시리얼 영상 스트리밍 (STEP 4-B 에서 추가)
 * - BLE 무선 송신 (STEP 4-C 에서 추가)
 * - OLED 화면 출력 (STEP 06, 제어 노드 측)
 *
 * 결과 확인: Arduino IDE 시리얼 모니터 (115200 baud)
 * 매 추론마다 RES: <label> (<conf>) <ms>ms
 * 안정화 통과 시 >>> STABLE: <label> (<conf>) <<<
 * ===== */

#include <object_clf_inferencing.h> // EI Studio에서 받은 라이브러리 헤더
// 학습된 모델 가중치, 라벨, run_classifier() 함수가 모두 들어 있음
#include "edge-impulse-sdk/dsp/image/image.hpp" // RGB565 -> RGB888 변환, 리사이즈 함수
#include "camera.h" // Arduino Nicla Camera 클래스
#include "gc2145.h" // GC2145 센서 드라이버
#include <ea_malloc.h> // mbed 환경의 정렬 메모리 할당

/* =====
 * 1. 설정값 (여기만 조정하면 동작이 바뀜)
 * ===== */
#define CONF_THRESHOLD 0.70f // 이 확신도 이상일 때만 결과로 인정 (0.0 ~ 1.0)
// 올리면 더 엄격 (놓치는 결과 늘고, 오인식 줄어들)
// 낮추면 더 관대 (반응 빠르고, 오인식 늘어남)
#define STABLE_REQUIRED 3 // 같은 답이 N번 연속이면 "확정"으로 판정
// 바늘 떨림 / OLED 깜빡임을 방지하는 핵심 장치
#define LOOP_INTERVAL_MS 100 // 추론 시도 간격(ms) - 실제 추론은 ~32ms 걸리므로
// 대략 100ms 주기 = 초당 10번 시도 (5fps급)

/* =====
 * 2. 카메라 글루 코드 (EI 공식 예제에서 무수정 이식)
 * =====
 * 이 영역은 GC2145 센서를 EI의 추론 파이프라인에 연결하는 "어댑터".
 * 학생 차시에서 직접 손댈 일은 거의 없고, 그대로 두고 동작.
 *
 * 동작 흐름:
 * 카메라 -> 320x240 RGB565 프레임 -> RGB888로 변환
 * -> 모델 입력 크기(64x64)로 리사이즈/크롭
 * -> EI 추론기에 픽셀 데이터 공급
 * ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320 // 카메라 출력 폭 (센서 고정)
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240 // 카메라 출력 높이
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE 2 // RGB565는 픽셀당 2바이트
// 32바이트 정렬 매크로 - DMA 전송 시 정렬되지 않으면 성능 저하 또는 오동작
#define ALIGN_PTR(p,a) ((p & (a-1)) ? (((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)

GC2145 galaxyCore; // GC2145 센서 드라이버 인스턴스
Camera cam(galaxyCore); // Arduino Camera API 래퍼
FrameBuffer fb; // 프레임 버퍼 객체

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL; // RGB888 변환/리사이즈 결과 버퍼
static uint8_t *ei_camera_frame_mem; // 원본 RGB565 프레임 메모리
static uint8_t *ei_camera_frame_buffer; // 32바이트 정렬된 포인터

typedef struct { size_t width; size_t height; } resize_t;

```

```

/* RGB565(2바이트/픽셀) -> RGB888(3바이트/픽셀) 변환
 * 카메라는 메모리 절약을 위해 RGB565를 출력하지만, 모델은 RGB888을 입력으로 받음 */
bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8; // R (상위 5비트)
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3; // G (중간 6비트)
        *dst++ = (lb & 0x1F) << 3; // B (하위 5비트)
    }
    return true;
}

/* 출력 해상도에 맞는 중간 리사이즈 크기 결정.
 * EI 라이브러리가 단계적으로 줄여나가는 방식을 사용 */
int calc_resize(uint32_t out_w, uint32_t out_h,
                uint32_t *col, uint32_t *row, bool *do_resize) {
    const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
    *col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
    *row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
    *do_resize = false;
    for (size_t i = 0; i < 5; i++) {
        if (out_w <= list[i].width && out_h <= list[i].height) {
            *col = list[i].width; *row = list[i].height; *do_resize = true; break;
        }
    }
    return 0;
}

/* 카메라 초기화 - setup()에서 한 번만 호출 */
bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;

    // 원본 RGB565 프레임 메모리 할당 (320 * 240 * 2 = 153,600 byte + 정렬 여유)
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;

    // 32바이트 정렬된 위치로 포인터 조정 (DMA 효율)
    ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
    fb.setBuffer(ei_camera_frame_buffer);
    is_initialised = true;
    return true;
}

/* 한 프레임 캡처 + RGB 변환 + 리사이즈
 * 호출 후 ei_camera_capture_out 에 모델 입력 크기로 준비된 RGB888 픽셀이 들어 있음 */
bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;

    // RGB888 출력 버퍼 할당 (320 * 240 * 3 = 230,400 byte + 정렬)
    // 함수 종료 시 ea_free로 반환 - 단편화 방지
    ei_camera_capture_out = (uint8_t*)ea_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);

    // 1) 카메라에서 RGB565 프레임 한 장 받기 (100ms 타임아웃)
    if (cam.grabFrame(fb, 100) != 0) return false;

    // 2) RGB888로 변환
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize()))
        return false;

    // 3) 모델 입력 크기로 리사이즈 + 중앙 크롭

```

```

uint32_t rc, rr; bool do_resize;
calc_resize(w, h, &rc, &rr, &do_resize);
if (do_resize) {
    ei::image::processing::crop_and_interpolate_rgb888(
        ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
        ei_camera_capture_out, rc, rr);
}
return true;
}

/* EI 추론기가 픽셀을 가져갈 때 호출하는 콜백.
 * RGB888 3바이트를 32비트 정수 하나로 패킹해 모델에 전달 */
static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
 * 3. 안정화 필터 상태 (전역 변수)
 * =====
 * 안정화 필터의 핵심 아이디어:
 * 카메라가 흔들리거나 빛이 변하면 모델이 잠깐 다른 답을 내기도 함.
 * -> "확신도 0.70 이상" + "같은 답이 3번 연속"을 모두 통과한 결과만
 * "STABLE = 확정"으로 인정. 그 외에는 무시.
 * 이 필터 덕분에 OLED 표시가 깜빡이지 않고, 서보 바늘이 떨지 않음.
 * ===== */
int lastClass = -1; // 직전 추론에서 가장 확신도 높았던 클래스
int stableCount = 0; // 같은 답이 몇 번 연속됐는지 카운트
int confirmedCls = -1; // 최근 "STABLE"로 확정된 클래스 (변화 시점 감지용)

/* =====
 * 4. setup() - 부팅 시 한 번만 실행
 * ===== */
void setup() {
    Serial.begin(115200);
    // USB 시리얼 연결을 5초간 기다림 (PC가 안 꽂혀 있어도 진행)
    while (!Serial && millis() < 5000) ;

    Serial.println();
    Serial.println("=====");
    Serial.println("Vision Gauge - STEP 4-A : Nicla 코어 테스트");
    Serial.println("=====");

    // [중요] Nicla Vision의 M4 코어 RAM 영역을 추가 할당 영역으로 등록.
    // 기본 M7 RAM(512KB)만으로는 EI TFLite arena가 부족해 부팅 직후 리셋됨.
    // 이 한 줄로 M4 영역에서 288KB를 추가 확보 - Arduino 공식 권장.
    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) {
        Serial.println("FATAL: 카메라 초기화 실패");
        while(1); // 무한 루프 (FATAL 표시 후 정지)
    }
    Serial.println("OK: 카메라 초기화 완료");

    // 학습된 모델 정보를 시리얼에 출력 - 학생이 자기 라벨이 맞는지 확인 가능
    Serial.print("모델 입력: ");
    Serial.print(EI_CLASSIFIER_INPUT_WIDTH); Serial.print("x");
    Serial.println(EI_CLASSIFIER_INPUT_HEIGHT);
    Serial.print("클래스 수: "); Serial.println(EI_CLASSIFIER_LABEL_COUNT);
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        Serial.print("  "); Serial.print(i); Serial.print(" ");

```



```

// 라벨은 EI Studio에서 자동으로 알파벳순으로 정렬됨
// STEP 06의 LABELS[] 배열 순서가 이 순서와 같아야 함
Serial.println(ei_classifier_inferencing_categories[i]);
}
Serial.print("필터: conf>="); Serial.print(CONF_THRESHOLD);
Serial.print(", stable="); Serial.println(STABLE_REQUIRED);
Serial.println("추론 루프 시작...");
Serial.println();
}

/* =====
* 5. loop() - 계속 반복 실행
* =====
* 매 LOOP_INTERVAL_MS(=100ms)마다:
* 1) 카메라 캡처 2) EI 추론 3) 가장 확신도 높은 클래스 선택
* 4) 시리얼에 결과 출력 5) 안정화 필터 적용
* ===== */
void loop() {
// 100ms 간격 유지 - 매 루프 즉시 추론하면 보드가 과열되고 USB도 막힘
static uint32_t lastTick = 0;
if (millis() - lastTick < LOOP_INTERVAL_MS) return;
lastTick = millis();

uint32_t t0 = millis(); // 추론 시간 측정 시작

// 1) 카메라 캡처
if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
ea_free(ei_camera_capture_out);
return;
}

// 2) 추론 - EI가 제공하는 signal_t 구조를 통해 픽셀 데이터를 모델에 전달
ei::signal_t signal;
signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
signal.get_data = &ei_camera_get_data; // 위에서 정의한 콜백

ei_impulse_result_t result = {0};
EI_IMPULSE_ERROR err = run_classifier(&signal, &result, false);
if (err != EI_IMPULSE_OK) {
Serial.print("ERR: run_classifier "); Serial.println(err);
ea_free(ei_camera_capture_out);
return;
}

// 3) argmax - 모델은 모든 클래스의 확률을 반환. 그중 가장 큰 것 하나만 선택
float maxConf = 0;
int maxIdx = -1;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
if (result.classification[i].value > maxConf) {
maxConf = result.classification[i].value;
maxIdx = i;
}
}

uint32_t totalMs = millis() - t0; // 캡처+추론 총 소요시간

// 4) 시리얼에 매 추론 결과 출력 (사람이 읽을 수 있도록 라벨명까지)
Serial.print("RES: ");
Serial.print(ei_classifier_inferencing_categories[maxIdx]);
Serial.print(" ("); Serial.print(maxConf, 2); Serial.print(") ");
Serial.print(totalMs); Serial.println("ms");

// 5) 안정화 필터 - "같은 답 N회 + conf >= 임계값" 통과 시에만 STABLE 확정
if (maxConf >= CONF_THRESHOLD) {
if (maxIdx == lastClass) {
stableCount++; // 같은 답 -> 카운트 증가
} else {

```

```

    stableCount = 1;          // 다른 답 -> 카운트 리셋
    lastClass = maxIdx;
}

// STABLE 통과 + "직전 확정과 다른 새 답"일 때만 STABLE 줄 출력
// (같은 답이 계속 STABLE로 들어와 줄이 도배되는 걸 방지)
if (stableCount >= STABLE_REQUIRED && maxIdx != confirmedCls) {
    confirmedCls = maxIdx;
    Serial.print(">>> STABLE: ");
    Serial.print(ei_classifier_inferencing_categories[confirmedCls]);
    Serial.print(" ("); Serial.print(maxConf, 2); Serial.println(") <<<");
}
} else {
    // 확신도가 임계값 미만 - "모름" 상태로 간주, 카운터 리셋
    // (이게 곧 unknown 클래스 없이도 "모름"을 처리하는 핵심 메커니즘)
    stableCount = 0;
    lastClass = -1;
}

ea_free(ei_camera_capture_out); // 캡처 버퍼 반환 - 단편화 방지
}

/* =====
 * 6. 안전 검증 - EI 라이브러리가 카메라 입력 모델인지 확인
 * ===== */
#if !defined(EI_CLASSIFIER_SENSOR) || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "이 펌웨어는 카메라 입력 모델 전용입니다 (EI 프로젝트 설정 확인)"
#endif

```

2. 시리얼 모니터(115200) 열기 → 다음 출력 확인 (아래 라벨은 예시 — 모두가 학습한 라벨로 다르게 표시됩니다):

```

=====
Vision Gauge — Nicla Core Test
=====
OK: camera initialized
Model input: 64x64
Classes (3):
  [0] glasses
  [1] keyboard
  [2] unkown
Filter: conf>=0.70, stable=3
Starting inference loop...

RES: unkown (0.45) 32ms
RES: glasses (0.88) 32ms
RES: glasses (0.91) 31ms
RES: glasses (0.89) 32ms
>>> STABLE: glasses (0.89) <<<

```

합격 체크리스트 (5개)

① 클래스 N개가 알파벳순 출력 ② 추론 시간 30~150ms ③ 시연 물체 conf 0.85+ ④ 물체 변경 시 ~0.6초 후 STABLE 한 번 ⑤ 5~10분 방치해도 멈춤 없음. 이 5개 모두 통과해야 4-B로 넘어갑니다.

학생 차시 빈발 이슈 — 4-A 영역

- ① 컴파일 에러 `object_clf_inferencing.h: No such file` : STEP 03-3 EI zip 라이브러리 미설치. 스케치 → 라이브러리 포함 → .ZIP 라이브러리 추가.
- ② 추론 시간 **130~140ms**: Quantized(int8)이 아니라 Unoptimized(float32)로 빌드된 케이스. EI Deployment에서 좌측 Quantized에 "Selected ✓" 확인 후 재빌드. 학생 차시 운영상 5fps여도 동작은 하니, 시간 부족 시 일단 진행하고 차시 후 재학습 권장.
- ③ 부팅 직후 무한 리셋: `malloc_addblock` 누락 (예제 코드에는 있음). 코드 변형 시 이 한 줄을 빠뜨리면 TFLite 영역 부족으로 죽습니다.
- ④ 업로드 직후 빨간 경고 `Invalid DFU suffix signature` : dfu-util의 미래 호환성 경고일 뿐. 바로 아래 `File downloaded successfully` 가 보이면 정상.
- ⑤ 시리얼 모니터에 한글 깨짐: baud 115200 확인. 펌웨어 재업로드 후 모니터 한 번 닫았다 열기.

4-B. USB 스트리밍 + 모니터 UI 레이어 추가

4-A에서 텍스트만 봤다면, 이제 PC 화면에서 영상과 함께 봅니다. 펌웨어에 줄 단위 프로토콜 3종을 추가하고, PC에는 Streamlit으로 모니터 UI를 띄웁니다.

4-B-1. 펌웨어 갱신

1. 아래 코드로 교체 업로드 — step1과 비교해 늘어난 부분은 `setup()` 의 LBL 출력, `loop()` 의 `if (Serial)` 가드 블록(RES/IMG 송신), base64 인코더 함수입니다:

코드 vision_node_step2_usb.ino — USB 스트리밍 추가

```
/* =====
 * Vision Gauge - STEP 4-B : USB 스트리밍 + 모니터 UI 레이어 추가
 *
 * step1과의 차이 (이 파일에서 새로 추가된 것):
 * [+ STEP 4-B] base64 인코더 함수 (serialPrintBase64)
 * [+ STEP 4-B] setup() 끝에 LBL: 라벨 목록 송신
 * [+ STEP 4-B] loop()의 USB 스트리밍 블록 (RES: / IMG: 줄)
 * [+ STEP 4-B] STABLE 시 사람용 로그 형식 변경 (# STABLE: ...)
 * [+ STEP 4-B] USB 미연결 시 송신 생략 가드 (if (Serial))
 *
 * step1과 동일한 것 (변경 없음):
 * - 카메라 클루, EI 추론, argmax, 안정화 필터 로직
 *
 * PC쪽 monitor.py가 받는 줄 단위 프로토콜:
 * LBL:<label1>,<label2>,... 부팅 시 1회 (학습한 라벨 목록)
 * RES:<idx>,<conf>,<t_ms> 매 추론 (가공 전 raw 결과)
 * IMG:<base64 64x64 RGB888> 3번에 1번 (모델이 본 영상)
 * # STABLE: <label> <conf> seq=<n> 안정화 확정 시
 * ===== */
```

```

#include <object_clf_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
#include "camera.h"
#include "gc2145.h"
#include <ea_malloc.h>

/* =====
 * 1. 설정값 (step1과 동일 + IMG 송신 빈도만 추가)
 * ===== */
#define CONF_THRESHOLD    0.70f
#define STABLE_REQUIRED   3
#define LOOP_INTERVAL_MS 100
#define IMG_EVERY_N       3    // [+ STEP 4-B] 3번 추론마다 1프레임 전송 (영상 부드러움)
                                //   매 추론마다 영상을 보내면 USB 대역폭 초과
                                //   IMG는 약 12KB 텍스트 - 5fps 정도가 적절

/* =====
 * 2. 카메라 클루 (step1과 100% 동일 - 변경 없음)
 * ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE   2
#define ALIGN_PTR(p,a) ((p & (a-1)) ? (((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p)

GC2145 galaxyCore;
Camera cam(galaxyCore);
FrameBuffer fb;

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL;
static uint8_t *ei_camera_frame_mem;
static uint8_t *ei_camera_frame_buffer;

typedef struct { size_t width; size_t height; } resize_t;

bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8;
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3;
        *dst++ = (lb & 0x1F) << 3;
    }
    return true;
}

int calc_resize(uint32_t out_w, uint32_t out_h,
                uint32_t *col, uint32_t *row, bool *do_resize) {
    const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
    *col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
    *row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
    *do_resize = false;
    for (size_t i = 0; i < 5; i++) {
        if (out_w <= list[i].width && out_h <= list[i].height) {
            *col = list[i].width; *row = list[i].height; *do_resize = true; break;
        }
    }
    return 0;
}

bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;

```

```

ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
fb.setBuffer(ei_camera_frame_buffer);
is_initialised = true;
return true;
}

bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;
    ei_camera_capture_out = (uint8_t*)ea_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);
    if (cam.grabFrame(fb, 100) != 0) return false;
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize())) return false;
    uint32_t rc, rr; bool do_resize;
    calc_resize(w, h, &rc, &rr, &do_resize);
    if (do_resize) {
        ei::image::processing::crop_and_interpolate_rgb888(
            ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
            ei_camera_capture_out, rc, rr);
    }
    return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
* [+ STEP 4-B 추가] 3. base64 인코더
* =====
* 왜 필요한가:
* 카메라 영상은 RGB888 바이너리 데이터 (값에 0x0A=개행, 0x00=NULL 등이 섞임).
* 바이너리를 그대로 시리얼로 보내면 줄 단위 파싱이 깨짐.
* -> base64로 변환하면 알파벳+숫자+/+ 만 사용해 텍스트 라인으로 안전 전송.
* 비용:
* 3바이트 -> 4글자로 늘어남 (33% 오버헤드). 64x64x3=12,288바이트 -> 16,384글자.
* ===== */
static const char b64chars[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

// 인코드 결과를 그대로 Serial로 흘려보냄 (메모리 절약 - 큰 버퍼 안 잡음)
void serialPrintBase64(const uint8_t *data, size_t len) {
    for (size_t i = 0; i < len; i += 3) {
        // 3바이트씩 묶어 24비트 정수 v로 합침
        uint32_t v = data[i] << 16;
        if (i + 1 < len) v |= data[i + 1] << 8;
        if (i + 2 < len) v |= data[i + 2];
        // 24비트를 6비트씩 4조각으로 잘라 base64 문자 인덱스로 사용
        Serial.print(b64chars[(v >> 18) & 0x3F]);
        Serial.print(b64chars[(v >> 12) & 0x3F]);
        Serial.print(i + 1 < len ? b64chars[(v >> 6) & 0x3F] : '='); // 패딩
        Serial.print(i + 2 < len ? b64chars[v & 0x3F] : '=');
    }
}

/* =====
* 4. 안정화 필터 상태 (step1과 동일 + 송신용 변수 추가)
* ===== */
int lastClass = -1;
int stableCount = 0;

```

```

int      confirmedCls = -1;
uint8_t  seqNum      = 0;  // [+ STEP 4-B] STABLE 시 sequence 번호 (PC측 ID)
uint32_t loopCount   = 0;  // [+ STEP 4-B] IMG 송신 빈도 카운터

/* [+ STEP 4-B] 라벨 목록 송신 함수
 * monitor.py가 이 LBL: 줄을 받아 "인덱스 -> 라벨명" 매핑을 만들.
 * 부팅 시 1회 + loop에서 주기적으로 재전송 (monitor가 나중에 연결돼도 라벨 수신) */
void sendLabels() {
    Serial.print("LBL:");
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        if (i) Serial.print(",");
        Serial.print(ei_classifier_inferencing_categories[i]);
    }
    Serial.println();
}

/* =====
 * 5. setup() - step1 + 라벨 송신 추가
 * ===== */
void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 5000) ;

    // [+ STEP 4-B] 시리얼 출력 형식 변경
    // step1: 사람 친화적인 긴 메시지
    // step2: monitor.py가 파싱하기 쉽도록 # 주석 + LBL/RES/IMG 줄 위주
    Serial.println("# Vision Gauge v2 - USB 스트리밍");

    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) { Serial.println("# FATAL: 카메라"); while(1); }

    // [+ STEP 4-B] 라벨 목록 송신 (sendLabels 함수로 분리 - loop에서 주기 재전송)
    sendLabels();
    Serial.println("# ready");
}

/* =====
 * 6. loop() - step1 추론 + [+ STEP 4-B] USB 스트리밍 블록
 * ===== */
void loop() {
    static uint32_t lastTick = 0;
    if (millis() - lastTick < LOOP_INTERVAL_MS) return;
    lastTick = millis();

    // [+ STEP 4-B] 라벨 목록 주기적 재전송 (5초마다)
    // monitor.py를 나중에 실행해도 라벨을 받을 수 있도록 - "#2" 대신 이름 표시
    static uint32_t lastLbl = 0;
    if (Serial && millis() - lastLbl > 5000) {
        lastLbl = millis();
        sendLabels();
    }

    uint32_t t0 = millis();
    if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
        ea_free(ei_camera_capture_out); return;
    }

    // 추론 (step1과 동일)
    ei::signal_t signal;
    signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
    signal.get_data      = &ei_camera_get_data;
    ei_impulse_result_t result = {0};
    if (run_classifier(&signal, &result, false) != EI_IMPULSE_OK) {
        ea_free(ei_camera_capture_out); return;
    }
}

```

```

// argmax (step1과 동일)
float maxConf = 0; int maxIdx = -1;
for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > maxConf) {
        maxConf = result.classification[i].value; maxIdx = i;
    }
}
uint32_t totalMs = millis() - t0;

/* ----- [+ STEP 4-B 추가] USB 송신 블록 -----
 * if (Serial) 가드:
 *   USB가 연결 안 됐을 때는 송신을 통째로 건너뛴.
 *   안 그러면 시리얼 버퍼가 꽉 차서 다음 추론이 늦어질 수 있음.
 *   (시연 시 PC 없이 보드만 동작시킬 때 중요)
 */
if (Serial) {
    // RES: 줄 - 매 추론마다 (가공 전 raw 결과)
    Serial.print("RES:");
    Serial.print(maxIdx); Serial.print(",");
    Serial.print(maxConf, 3); Serial.print(",");
    Serial.println(totalMs);

    // IMG: 줄 - 3번에 1번 (모델이 본 영상을 base64 텍스트로)
    if (LoopCount % IMG_EVERY_N == 0) {
        Serial.print("IMG:");
        size_t pixBytes = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT * 3;
        serialPrintBase64(ei_camera_capture_out, pixBytes);
        Serial.println();
    }
}
LoopCount++;

// 안정화 필터 (step1과 거의 동일, STABLE 출력 형식만 변경)
if (maxConf >= CONF_THRESHOLD) {
    if (maxIdx == lastClass) stableCount++;
    else { stableCount = 1; lastClass = maxIdx; }

    if (stableCount >= STABLE_REQUIRED && maxIdx != confirmedCls) {
        confirmedCls = maxIdx;
        seqNum++;
        // [+ STEP 4-B] 사람용 로그 형식 - monitor.py가 "# STABLE:"으로 파싱
        //   PC쪽 모니터에서 "확정 결과" 박스를 갱신하는 트리거
        Serial.print("# STABLE: ");
        Serial.print(ei_classifier_inferencing_categories[confirmedCls]);
        Serial.print(" "); Serial.print(maxConf, 2);
        Serial.print(" seq="); Serial.println(seqNum);
        // [+ STEP 4-C 예정] 여기서 BLE notify로 3바이트 송신 (step3에서 추가)
    }
} else {
    stableCount = 0; lastClass = -1;
}

ea_free(ei_camera_capture_out);
}

#ifdef EI_CLASSIFIER_SENSOR || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "이 펌웨어는 카메라 입력 모델 전용입니다"
#endif

```

줄	의미	빈도
LBL:<라벨1,2,...>	라벨 동기화 (PC가 인덱스→이름 자동 매핑)	부팅 1회
RES:<idx>,<conf>,<t_ms>	매 추론 raw 결과 (안정화 전)	매 추론
IMG:<base64 64×64 RGB888>	모델이 본 영상 (약 12KB/장 텍스트)	3번에 1번
# STABLE: ...	사람용 로그, 안정화 확정 시	변화 시

2. 시리얼 모니터로 잠깐 확인 — 위 4종이 흐르면 성공. **확인 후 시리얼 모니터는 반드시 닫기.** 같은 COM 포트를 monitor.py가 열어야 합니다.
3. **중요** — 펌웨어에 `if (Serial) { ... }` 가드가 들어 있어, USB가 빠진 상태에서는 IMG/RES 송신을 건너뜁니다. 시연 시 USB 뽑아도 보드는 정상 동작 (4-C 단계의 BLE만 송신).

4-B-2. PC 모니터 UI — uv 가상환경 + Streamlit

1. PowerShell에서 uv 설치 (한 번만):

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

설치 후 모든 PowerShell 창을 닫고 새로 엽니다 (PATH 갱신).

2. 프로젝트 폴더 + 패키지:

```
cd $HOME
mkdir vision_gauge
cd vision_gauge
uv init
uv add streamlit pyserial numpy pillow
```

주의 — System32에서 mkdir 시 권한 거부. 반드시 `cd $HOME` 먼저.

3. 아래 코드를 `vision_gauge/monitor.py` 로 저장. 백그라운드 스레드로 시리얼을 읽고 머리말로 분기하는 구조 — 줄을 추가하려면 reader_loop의 분기만 늘리면 됩니다.


```

"""
=====
Vision Gauge - 코어측 모니터 UI (STEP 4-B / STEP 05)
-----
Nicla Vision의 USB 시리얼을 읽어 PC 화면에 표시하는 Streamlit 앱.

Nicla Vision이 보내는 줄 단위 프로토콜:
  LBL:<라벨1>,<라벨2>,...   부팅 1회 (학습한 라벨 목록)
  RES:<idx>,<conf>,<t_ms>   매 추론 (raw 결과)
  IMG:<base64 64x64 RGB888> 3번에 1번 (모델이 본 영상)
  # STABLE: <라벨> <conf> seq=<n>   안정화 확정 시
  # ...                               그 외 주석 (디버그용, UI에는 미사용)

화면 구성:
  좌측 - 64x64 영상 (5배 확대)
  우측 - 실시간 인식 카드 (클래스별 색상, 큰 폰트, 퍼센트)
  하단 - 최근 20개 이력 표

실행:
  uv run streamlit run monitor.py
=====
"""

import base64
import io
import threading
import time
from collections import deque
from dataclasses import dataclass, field

import numpy as np
import serial
import serial.tools.list_ports
import streamlit as st
from PIL import Image

# ----- 상수 -----
IMG_W, IMG_H = 64, 64      # 모델 입력 해상도 (펌웨어와 일치해야 함)
BAUD = 115200              # 시리얼 통신 속도 (펌웨어 Serial.begin과 같아야)
POLL_MS = 150              # UI 갱신 주기 (150ms = 초당 약 6.7회, 영상 부드러움)

# 클래스별 색상 팔레트 - 다크 배경에서 서로 명확히 대비되는 테마 컬러맵
# (Tableau 10 계열을 다크 테마에 맞게 밝기 보정 - 인접 색이 확연히 구분됨)
CLASS_COLORS = [
    "#4FC3F7", # 0 하늘색
    "#FF7043", # 1 주황
    "#66BB6A", # 2 초록
    "#BA68C8", # 3 보라
    "#FFCA28", # 4 노랑
    "#EC407A", # 5 핑크
    "#26C6DA", # 6 청록
    "#9CCC65", # 7 연두
]

def class_color(idx: int) -> str:
    """클래스 인덱스 -> 색상 (개수 초과 시 순환)"""
    return CLASS_COLORS[idx % len(CLASS_COLORS)]

def semicircle_gauge(pct: int, color: str) -> str:
    """반원형(180도) 게이지 SVG 생성.
    pct: 0~100 확신도, color: 채움 색상.
    바늘이 좌(0%)에서 우(100%)로 회전, 중심축 하단에 Confidence 표기."""

```

```

import math
# 반원: 왼쪽(180도)에서 오른쪽(0도)으로. pct에 따라 바늘 각도 계산
# 180도(왼쪽 끝) ~ 0도(오른쪽 끝)
angle_deg = 180 - (pct / 100) * 180
angle_rad = math.radians(angle_deg)
cx, cy, r = 130, 130, 100 # 중심, 반지름
# 바늘 끝점
nx = cx + r * 0.82 * math.cos(angle_rad)
ny = cy - r * 0.82 * math.sin(angle_rad)

# 호(arc) 경로 - 배경(회색 전체 반원) + 채움(색상, pct만큼)
def arc_point(p):
    a = math.radians(180 - (p / 100) * 180)
    return (cx + r * math.cos(a), cy - r * math.sin(a))
bg_start = arc_point(0)
bg_end = arc_point(100)
fill_end = arc_point(pct)
large = 0 # 반원이므로 항상 0
sweep_bg = 1
# 채움 호: 0%(왼쪽)에서 pct까지
fill_path = (f"M {bg_start[0]:.1f} {bg_start[1]:.1f} "
             f"A {r} {r} 0 0 1 {fill_end[0]:.1f} {fill_end[1]:.1f}")
bg_path = (f"M {bg_start[0]:.1f} {bg_start[1]:.1f} "
           f"A {r} {r} 0 0 1 {bg_end[0]:.1f} {bg_end[1]:.1f}")

return f"""
<svg viewBox="0 0 260 170" style="width:100%;max-width:340px;">
  <!-- 배경 호 (회색) -->
  <path d="{bg_path}" fill="none" stroke="#1e293b" stroke-width="20"
        stroke-linecap="round"/>
  <!-- 채움 호 (색상) -->
  <path d="{fill_path}" fill="none" stroke="{color}" stroke-width="20"
        stroke-linecap="round"/>
  <!-- 바늘 -->
  <line x1="{cx}" y1="{cy}" x2="{nx:.1f}" y2="{ny:.1f}"
        stroke="#f1f5f9" stroke-width="4" stroke-linecap="round"/>
  <!-- 중심축 -->
  <circle cx="{cx}" cy="{cy}" r="9" fill="#f1f5f9"/>
  <circle cx="{cx}" cy="{cy}" r="4" fill="{color}"/>
  <!-- 퍼센트 숫자 (호 안쪽 상단, 바늘과 겹치지 않게) -->
  <text x="{cx}" y="78" text-anchor="middle" fill="#f1f5f9"
        font-size="36" font-weight="800">{pct}<tspan font-size="18" fill="#94a3b8">%</tspan></text>
  <!-- Confidence 라벨 (중심축 하단) -->
  <text x="{cx}" y="158" text-anchor="middle" fill="#94a3b8"
        font-size="15" font-weight="600" letter-spacing="0.1em">Confidence</text>
</svg>"""

```

```

# =====
# 1. 백그라운드 스레드용 공유 상태
# -----
# 시리얼 읽기는 블로킹 작업이므로 별도 스레드에서 처리.
# Streamlit 메인 스레드는 이 State를 읽어 화면만 그림.
# =====

@dataclass
class State:
    labels: list = field(default_factory=list) # LBL:로 받은 라벨 목록
    last_img: np.ndarray = None # 마지막 IMG: 디코드 결과
    last_res: dict = None # 최근 RES: {"label", "conf", "t_ms"}
    history: deque = field(default_factory=lambda: deque(maxlen=20)) # 이력
    stable: dict = None # 최근 STABLE 결과
    connected: bool = False # 시리얼 연결 상태
    error: str = "" # 에러 메시지
    ble_tx: str = "idle" # BLE 송신 상태: idle/sent/waiting
    ble_tx_time: float = 0.0 # 마지막 BLE 송신 시각

```

```

# =====
# 2. 시리얼 읽기 백그라운드 루프
# -----
# - PySerial로 한 줄씩 읽기
# - 머리말(LBL:/RES:/IMG:/# STABLE:)로 분기해 State 갱신
# - stop_flag로 종료 신호 받음
# =====
def reader_loop(port: str, state: State, stop_flag: list):
    try:
        ser = serial.Serial(port, BAUD, timeout=1)
        state.connected = True
        state.error = ""
    except Exception as e:
        state.error = f"포트 열기 실패: {e}"
        return

    try:
        while not stop_flag[0]:
            try:
                line = ser.readline().decode("utf-8", errors="ignore").strip()
            except Exception:
                continue
            if not line:
                continue

            # ---- LBL: 라벨 목록 (부팅 1회) ----
            if line.startswith("LBL:"):
                state.labels = line[4:].split(",")

            # ---- RES: 매 추론 결과 ----
            elif line.startswith("RES:"):
                try:
                    idx_str, conf_str, ms_str = line[4:].split(",")
                    idx = int(idx_str)
                    conf = float(conf_str)
                    t_ms = int(ms_str)
                    label = state.labels[idx] if 0 <= idx < len(state.labels) else f"#{idx}"
                    state.last_res = {"label": label, "conf": conf, "t_ms": t_ms, "idx": idx}
                    state.history.append({
                        "시각": time.strftime("%H:%M:%S", time.localtime()),
                        "라벨": label,
                        "확신도": f"{int(round(conf*100))}%",
                    })
                except ValueError:
                    pass # 파싱 실패는 무시 (잡음 줄)

            # ---- IMG: 영상 base64 ----
            elif line.startswith("IMG:"):
                try:
                    raw = base64.b64decode(line[4:])
                    # 길이 검증 - 줄이 잘리지 않았는지 확인
                    if len(raw) == IMG_W * IMG_H * 3:
                        arr = np.frombuffer(raw, dtype=np.uint8).reshape(IMG_H, IMG_W, 3)
                        state.last_img = arr
                except Exception:
                    pass

            # ---- # STABLE: 안정화 확정 ----
            # 예: "# STABLE: bottle 0.89 seq=3"
            elif line.startswith("# STABLE:"):
                try:
                    parts = line[len("# STABLE:"):].strip().split()
                    label = parts[0]
                    conf = float(parts[1])
                    # 라벨 -> 인덱스 (색상 매핑용)
                    idx = state.labels.index(label) if label in state.labels else 0
                    state.stable = {"label": label, "conf": conf, "idx": idx,

```

```

        "time": time.strftime("%H:%M:%S")}

    except Exception:
        pass

    # ---- BLE 송신 상태 (step3가 # TX: 줄로 출력) ----
    # "... (BLE central에 notify 전송)" -> 송신 성공
    # "... (BLE 미연결 ...)" -> 수신자 없음
    elif "BLE central" in line and "notify" in line:
        state.ble_tx = "sent"
        state.ble_tx_time = time.time()
    elif "BLE 미연결" in line:
        state.ble_tx = "waiting"
        state.ble_tx_time = time.time()

    # 그 외 "# ..." 주석 줄은 무시 (디버그용)
finally:
    ser.close()
    state.connected = False

# =====
# 3. Streamlit UI - 메인 스레드
# =====
st.set_page_config(page_title="Vision Gauge Monitor", layout="wide")
st.title("Vision Gauge Monitor")
st.caption("Nicla Vision의 USB 시리얼 출력을 실시간으로 확인")

# ---- 사이드바: 포트 선택 + 연결 토글 ----
with st.sidebar:
    st.subheader("연결")
    ports = [p.device for p in serial.tools.list_ports.comports()]
    if not ports:
        st.warning("시리얼 포트를 찾을 수 없습니다. 보드가 USB로 연결되었는지 확인하세요.")
        st.stop()

    selected_port = st.selectbox("포트", ports)

    if "thread" not in st.session_state:
        st.session_state.thread = None
        st.session_state.state = State()
        st.session_state.stop_flag = [False]

    if st.button("연결" if not st.session_state.state.connected else "연결 끊기"):
        if st.session_state.thread is None or not st.session_state.thread.is_alive():
            # 시작
            st.session_state.state = State()
            st.session_state.stop_flag = [False]
            t = threading.Thread(
                target=reader_loop,
                args=(selected_port, st.session_state.state, st.session_state.stop_flag),
                daemon=True,
            )
            t.start()
            st.session_state.thread = t
        else:
            # 종료
            st.session_state.stop_flag[0] = True
            st.session_state.thread = None

# 상태 표시
state: State = st.session_state.state
if state.connected:
    st.success(f"연결됨: {selected_port}")
elif state.error:
    st.error(state.error)
else:
    st.info("대기 중")

```

```

# ---- BLE 송신 상태 인디케이터 ----
# 비전 노드가 STABLE 시 BLE notify를 보내는지 USB 로그로 추적
st.markdown("***BLE 송신**")
ble_fresh = (time.time() - state.ble_tx_time) < 3.0 # 3초 내 활동
if state.ble_tx == "sent" and ble_fresh:
    st.success("    송신 중 (제어 노드 수신)")
elif state.ble_tx == "waiting" and ble_fresh:
    st.warning("⚠ 수신자 없음 (제어 노드 미연결)")
else:
    st.caption("🕒 대기 — STABLE 확정 시 송신")

if state.labels:
    st.markdown("***라벨**")
    for i, lab in enumerate(state.labels):
        st.write(f"`[{i}]` {lab}")

# ---- 메인 영역: 영상 + 결과 (둘 다 정사각형, 1:1 배치) ----
col_img, col_res = st.columns([1, 1])

with col_img:
    st.subheader("모델이 보는 영상")
    img_area = st.empty()

with col_res:
    st.subheader("실시간 인식 결과")
    res_area = st.empty()

st.subheader("최근 20개 이력")
hist_area = st.empty()

# ---- POLL_MS마다 화면 갱신 (67회 x 150ms 약 10초 후 rerun) ----
# Streamlit은 자동 새로고침이 없으므로 명시적으로 st.rerun() 호출
POLL_MS_total = POLL_MS / 1000
for _ in range(67):
    state: State = st.session_state.state

    # 영상 - 정사각형(aspect-ratio 1:1). base64로 HTML img에 삽입
    if state.last_img is not None:
        img = Image.fromarray(state.last_img).resize((IMG_W * 5, IMG_H * 5),
                                                    Image.NEAREST)

        buf = io.BytesIO()
        img.save(buf, format="PNG")
        b64 = base64.b64encode(buf.getvalue()).decode()
        img_area.markdown(f"""
<div style="aspect-ratio:1/1;border-radius:18px;overflow:hidden;margin-top:8px;
background:#000;display:flex;align-items:center;justify-content:center;">

</div>
""", unsafe_allow_html=True)
    else:
        img_area.markdown("""
<div style="aspect-ratio:1/1;border-radius:18px;margin-top:8px;background:#1e293b;
display:flex;align-items:center;justify-content:center;
color:#94a3b8;font-size:16px;">영상 대기 중...</div>
""", unsafe_allow_html=True)

    # ---- 실시간 인식 (영상과 같은 높이의 큰 카드, 클래스별 색상) ----
    if state.last_res:
        r = state.last_res
        # 라벨을 아직 못 받아 인덱스(#N)로 표시되는 경우 - 동기화 안내
        if r["label"].startswith("#"):
            res_area.markdown("""
<div style="background:#1e293b;border:2px dashed #475569;border-radius:18px;
box-sizing:border-box;padding:40px 32px;margin-top:8px;text-align:center;

```

```

        display:flex;flex-direction:column;justify-content:center;align-items:center;
        aspect-ratio:1/1;">
<div style="font-size:22px;color:#94a3b8;">라벨 동기화 중...</div>
<div style="font-size:15px;color:#64748b;margin-top:10px;">
    비전 노드에서 라벨 목록을 받는 중입니다 (몇 초 소요)</div>
</div>
""" , unsafe_allow_html=True)
    else:
        color = class_color(r["idx"])
        pct = int(round(r["conf"] * 100))
        # 절충안(실시간 송신): STABLE 개념 대신 확신도로 상태 구분
        if pct >= 70:
            check = "● 안정적 인식"
        elif pct >= 50:
            check = "● 인식 중"
        else:
            check = "○ 탐색 중"

        gauge = semicircle_gauge(pct, color)
        res_area.markdown(f"""
<div style="background:linear-gradient(135deg,{color}26,{color}0a);
border:2px solid {color};border-radius:18px;box-sizing:border-box;
padding:28px 32px;margin-top:8px;aspect-ratio:1/1;
display:flex;flex-direction:column;align-items:center;
justify-content:center;text-align:center;">
<div style="font-size:16px;color:{color};letter-spacing:0.08em;
font-weight:600;margin-bottom:10px;">{check}</div>
<div style="font-size:60px;font-weight:800;color:{color};
line-height:1.05;margin-bottom:8px;
word-break:break-all;text-shadow:0 2px 12px {color}40;">{r['label']}</div>
{gauge}
</div>
""" , unsafe_allow_html=True)
    else:
        res_area.markdown("""
<div style="background:#1e293b;border:2px dashed #475569;border-radius:18px;
box-sizing:border-box;padding:40px 32px;margin-top:8px;text-align:center;
display:flex;flex-direction:column;justify-content:center;align-items:center;
aspect-ratio:1/1;">
<div style="font-size:22px;color:#94a3b8;">추론 결과 대기 중...</div>
</div>
""" , unsafe_allow_html=True)

    # 이력 표
    if state.history:
        rows = list(state.history)[::-1]
        hist_area.dataframe(rows, use_container_width=True, hide_index=True)
    else:
        hist_area.empty()

    time.sleep(POLL_MS_total)

st.rerun() # 약 10초마다 페이지 자동 새로고침 - Streamlit 특성상 필요

```

4. 실행:

```
uv run streamlit run monitor.py
```

주의 — `python monitor.py` 로는 안 됩니다. 두 가지 이유: ① 시스템 파이썬에는 `numpy/streamlit`이 없어 `ModuleNotFoundError`, ② `Streamlit` 앱은 전용 실행기로 띄워야 브라우저 UI가 동작.

5. 브라우저가 자동으로 열림. 안 열리면 콘솔의 `http://localhost:8501` 직접 접속.

운영 편의 — 배치파일(.bat)로 더블클릭 실행

학생·교사가 매번 터미널에서 폴더로 이동해 명령을 입력하는 부담을 없애려면, 배치파일을 만들어 배포하는 것이 좋습니다. 메모장에 아래 4줄을 입력하고 **"모든 파일(*.*)"** 형식 / **UTF-8 인코딩** / **대시보드_실행.bat** 이름으로 `monitor.py` 폴더에 저장합니다.

```
@echo off
cd /d "%~dp0"
uv run streamlit run monitor.py
pause
```

각 줄의 역할 — `@echo off` (명령어 표시 끄기) · `cd /d "%~dp0"` (`%~dp0` =배치파일 자신의 폴더 경로, `/d` =드라이브가 달라도 이동) · 실행 명령 · `pause` (오류 시 창 유지로 메시지 확인). 핵심은 `%~dp0` 로 경로를 하드코딩하지 않는 것입니다 — 폴더를 옮기거나 다른 PC에 복사해도 그대로 동작하므로, 학생 PC마다 경로를 고칠 필요가 없습니다.

바탕화면 바로가기: .bat 우클릭 → **"추가 옵션 표시"**(Windows 11에서 단순 메뉴일 때) → **"보내기"** → **"바탕 화면(바로 가기 만들기)"**. 원본 .bat은 폴더에 그대로 두어야 하며(바로가기는 원본 위치 기준 동작), 한글 깨짐이 있으면 첫 줄 다음에 `chcp 65001 >nul` 을 추가하면 콘솔이 UTF-8로 바뀝니다.

6. 사이드바에서 COM 포트 선택 → 연결 → 1초 후 상태 연결됨

`monitor.py` 핵심 동작

백그라운드 스레드가 시리얼을 줄 단위로 읽다가 머리말로 분기 — `LBL:` 은 라벨 리스트, `RES:` 는 게이지·이력 갱신, `IMG:` 는 base64 → `numpy` → 영상 표시, `# STABLE:` 은 초록 박스 갱신. `Streamlit`은 10초마다 `st.rerun()`으로 새로 그리되 폴링 주기는 200ms.

- ① **ModuleNotFoundError: No module named 'numpy'** : `python monitor.py` 로 실행한 경우. 반드시 `uv run streamlit run monitor.py` .
- ② **사이드바 포트 목록에 COM 없음**: Arduino IDE 시리얼 모니터가 열려 있어 점유 중. 닫고 페이지 새로 고침.
- ③ **"포트 열기 실패"**: 같은 원인이거나, 펌웨어 업로드 직후 보드 재부팅 중. 2~3초 후 재연결.
- ④ **영상이 안 뜨고 RES만 옴**: 펌웨어가 v1(step1_test)인 상태. v2(step2_usb)로 재업로드 확인.
- ⑤ **영상이 깨져 보임 / 색이 이상함**: base64 디코드 한 번 어긋난 경우. monitor.py에서 ■ 정지 → 재 연결로 회복.
- ⑥ **Streamlit 페이지가 자꾸 깜빡임**: 정상입니다. 10초 주기 rerun으로 화면을 새로 그립니다.

4-C. BLE 송신 레이어 추가

4-B까지로 "PC 모니터에서 보는 엣지 AI"가 완성되었습니다. 이제 USB가 없어도 동작하는 무선 채널을 추가 — 최신 추론 결과를 200ms마다 3바이트로 압축해 실시간 BLE 송신. STEP 06의 OLED 제어 노드가 이 신호를 받아 화면에 표시합니다.

4-C-1. 라이브러리·라이브러리 호환성 안내

- Arduino IDE 라이브러리 매니저 → **"ArduinoBLE"** 설치 (Arduino 공식, 현재 v1.3.x)
- 이 라이브러리는 mbed 계열 보드용 — Nicla Vision은 정상. 제어 노드 ESP32는 다른 라이브러리 ([BLEDevice.h](#)) 사용.
- 전파 규격(BLE 4.x)은 같지만 **코드 문법이 라이브러리별로 달라 서로 복불 불가**합니다. 학생 차시에 자주 헷갈리는 부분 — Nicla 코드와 ESP32 코드를 색이 다른 카드로 구분해 안내하면 좋습니다.

4-C-2. 펌웨어 (step2 → step3 변경점)

위치	추가 내용	이유
#include	<ArduinoBLE.h>	BLE Peripheral API 사용
전역	BLEService + BLECharacteristic(3byte, Read+Notify)	UUID 두 개 정의, 특성은 3바이트 고정
setup()	BLE.begin() → setLocalName → setAdvertisedService → addCharacteristic → addService → advertise()	표준 advertise 5단계 — 순서 바꾸면 동작 안 함
loop() 첫 줄	BLE.poll();	★ 가장 흔한 누락 — 빠지면 연결 직후 1~2초 만에 끊김
200ms 주기	resultChar.writeValue(buf, 3)	최신 결과 실시간 송신 (절충안). BLE 미연결 시는 로컬 값만 갱신

코드 vision_node_step3_ble.ino — BLE 송신 레이어 추가

```

/* =====
 * Vision Gauge - STEP 4-C : BLE 송신 레이어 추가
 *
 * step2와의 차이 (이 파일에서 새로 추가된 것):
 * [+ STEP 4-C] #include <ArduinoBLE.h>
 * [+ STEP 4-C] BLE 서비스/특성 객체 전역 선언 (결과 + 라벨)
 * [+ STEP 4-C] setup()에서 BLE.begin -> advertise (5단계)
 * [+ STEP 4-C] loop() 첫 줄 BLE.poll() (생략 시 끊김!)
 * [+ STEP 4-C] 200ms마다 최신 추론 결과를 3바이트 notify 송신 (실시간)
 *
 * step2와 동일한 것 (변경 없음):
 * - 카메라 글루, EI 추론, argmax
 * - USB 스트리밍 (LBL/RES/IMG)
 *
 * BLE 송신 방식 (절충안 - 실시간):
 * - 안정화 필터(STABLE) 없이 매 추론의 최신 결과를 200ms마다 송신.
 * - 확신도가 낮아도 즉시 반영되고, 200ms 쓰로틀로 OLED 깜빡임을 억제.
 *
 * BLE 송신 데이터:
 * 3바이트 [class_id, conf*100, seq]
 * class_id : 클래스 인덱스 (0~N-1, 알파벳순)
 * conf*100 : 확신도 정수 (0~100, 0.85 -> 85)
 * seq      : 시퀀스 번호 (0~255 순환, 송신마다 증가)
 *
 * 검증 절차:
 * 1) USB 시리얼로 RES/IMG/TX 줄이 흐르는지 (4-B + TX)
 * 2) 폰 nRF Connect 앱으로 "VisionGauge" 검색/연결
 * 3) 0x7e400002 특성 구독 후 물체 변경 시 3바이트 갱신 확인
 * ===== */

#include <object_clf_inferencing.h>
#include "edge-impulse-sdk/dsp/image/image.hpp"
#include "camera.h"

```

```

#include "gc2145.h"
#include <ea_malloc.h>
#include <ArduinoBLE.h>                                // [+ STEP 4-C] BLE Peripheral API

/* =====
 * 1. 설정값
 * ===== */
#define LOOP_INTERVAL_MS 100    // 추론 시도 간격(ms)
#define IMG_EVERY_N 3          // IMG 송신 빈도 (3번에 1번)
#define BLE_TX_INTERVAL_MS 200 // [절충안] BLE 송신 주기 - 200ms마다 최신 결과
                                // 짧으면 반응 빠르나 OLED 깜빡임/대역폭 ↑
                                // 길면 부드러우나 반응 느림. 200ms가 균형점

/* =====
 * [+ STEP 4-C 추가] 2. BLE 서비스/특성 정의
 * -----
 * UUID는 임의의 128비트 값 - "이 서비스는 다른 BLE 기기와 충돌하지 않는다"
 * 는 의미만 가짐. 0x7e40 prefix는 우리 프로젝트 고유 식별자로 사용.
 * 제어 노드(ESP32)의 코드도 이 UUID와 정확히 일치해야 함.
 * ===== */
#define BLE_DEVICE_NAME "VisionGauge" // nRF Connect 스캔에 표시될 이름
#define SERVICE_UUID "7e400001-b5a3-f393-e0a9-e50e24dcca9e"
#define CHAR_UUID "7e400002-b5a3-f393-e0a9-e50e24dcca9e"
#define LABEL_UUID "7e400003-b5a3-f393-e0a9-e50e24dcca9e" // [+ 라벨 특성]

BLEService visionService(SERVICE_UUID);
// 결과 특성 - 3바이트 [class_id, conf*100, seq], 자주 갱신
// BLERead : Central이 값을 읽을 수 있음
// BLENotify : 값이 바뀌면 Central에 자동 푸시 (구독 시)
BLECharacteristic resultChar(CHAR_UUID, BLERead | BLENotify, 3);

// [+ 라벨 특성] 라벨 목록 문자열 "label1,label2,unknown"
// 부팅 시 1회 기록, 제어 노드가 연결 직후 read해서 LABELS[] 채움.
// 최대 64바이트 (라벨이 길거나 많으면 늘릴 것)
BLECharacteristic labelChar(LABEL_UUID, BLERead, 64);

/* =====
 * 3. 카메라 글루 (step1/step2와 100% 동일)
 * ===== */
#define EI_CAMERA_RAW_FRAME_BUFFER_COLS 320
#define EI_CAMERA_RAW_FRAME_BUFFER_ROWS 240
#define EI_CAMERA_RAW_FRAME_BYTE_SIZE 2
#define ALIGN_PTR(p,a) ((p & (a-1)) ? ((uintptr_t)p + a) & ~(uintptr_t)(a-1)) : p

GC2145 galaxyCore;
Camera cam(galaxyCore);
FrameBuffer fb;

static bool is_initialised = false;
static uint8_t *ei_camera_capture_out = NULL;
static uint8_t *ei_camera_frame_mem;
static uint8_t *ei_camera_frame_buffer;

typedef struct { size_t width; size_t height; } resize_t;

bool RGB565ToRGB888(uint8_t *src, uint8_t *dst, uint32_t src_len) {
    uint32_t pix = src_len / 2;
    for (uint32_t i = 0; i < pix; i++) {
        uint8_t hb = *src++, lb = *src++;
        *dst++ = hb & 0xF8;
        *dst++ = (hb & 0x07) << 5 | (lb & 0xE0) >> 3;
        *dst++ = (lb & 0x1F) << 3;
    }
    return true;
}

int calc_resize(uint32_t out_w, uint32_t out_h,

```

```

        uint32_t *col, uint32_t *row, bool *do_resize) {
const resize_t list[] = {{64,64},{96,96},{160,120},{160,160},{320,240}};
*col = EI_CAMERA_RAW_FRAME_BUFFER_COLS;
*row = EI_CAMERA_RAW_FRAME_BUFFER_ROWS;
*do_resize = false;
for (size_t i = 0; i < 5; i++) {
    if (out_w <= list[i].width && out_h <= list[i].height) {
        *col = list[i].width; *row = list[i].height; *do_resize = true; break;
    }
}
return 0;
}

bool ei_camera_init() {
    if (is_initialised) return true;
    if (!cam.begin(CAMERA_R320x240, CAMERA_RGB565, -1)) return false;
    ei_camera_frame_mem = (uint8_t*)ei_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS
        * EI_CAMERA_RAW_FRAME_BYTE_SIZE + 32);
    if (!ei_camera_frame_mem) return false;
    ei_camera_frame_buffer = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_frame_mem, 32);
    fb.setBuffer(ei_camera_frame_buffer);
    is_initialised = true;
    return true;
}

bool ei_camera_capture(uint32_t w, uint32_t h) {
    if (!is_initialised) return false;
    ei_camera_capture_out = (uint8_t*)ea_malloc(
        EI_CAMERA_RAW_FRAME_BUFFER_COLS * EI_CAMERA_RAW_FRAME_BUFFER_ROWS * 3 + 32);
    ei_camera_capture_out = (uint8_t*)ALIGN_PTR((uintptr_t)ei_camera_capture_out, 32);
    if (cam.grabFrame(fb, 100) != 0) return false;
    if (!RGB565ToRGB888(ei_camera_frame_buffer, ei_camera_capture_out, cam.frameSize())) return false;
    uint32_t rc, rr; bool do_resize;
    calc_resize(w, h, &rc, &rr, &do_resize);
    if (do_resize) {
        ei::Image::processing::crop_and_interpolate_rgb888(
            ei_camera_capture_out, EI_CAMERA_RAW_FRAME_BUFFER_COLS, EI_CAMERA_RAW_FRAME_BUFFER_ROWS,
            ei_camera_capture_out, rc, rr);
    }
    return true;
}

static int ei_camera_get_data(size_t offset, size_t length, float *out_ptr) {
    size_t pix_ix = offset * 3;
    for (size_t i = 0; i < length; i++) {
        out_ptr[i] = (ei_camera_capture_out[pix_ix] << 16)
            + (ei_camera_capture_out[pix_ix + 1] << 8)
            + ei_camera_capture_out[pix_ix + 2];
        pix_ix += 3;
    }
    return 0;
}

/* =====
* 4. base64 인코더 (step2에서 추가, 그대로 유지)
* ===== */
static const char b64chars[] =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

void serialPrintBase64(const uint8_t *data, size_t len) {
    for (size_t i = 0; i < len; i += 3) {
        uint32_t v = data[i] << 16;
        if (i + 1 < len) v |= data[i + 1] << 8;
        if (i + 2 < len) v |= data[i + 2];
        Serial.print(b64chars[(v >> 18) & 0x3F]);
        Serial.print(b64chars[(v >> 12) & 0x3F]);
    }
}

```

```

        Serial.print(i + 1 < len ? b64chars[(v >> 6) & 0x3F] : '=');
        Serial.print(i + 2 < len ? b64chars[ v          & 0x3F] : '=');
    }
}

/* =====
 * 5. 송신 상태 변수
 * ===== */
uint8_t seqNum      = 0; // BLE 송신 시퀀스 번호 (0~255 순환)
uint32_t loopCount  = 0; // IMG 송신 빈도 카운터

/* 라벨 목록 USB 송신 함수 (부팅 1회 + loop 주기 재전송)
 * monitor.py가 나중에 연결돼도 라벨을 받아 "#2" 대신 이름 표시 */
void sendLabels() {
    Serial.print("LBL:");
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        if (i) Serial.print(",");
        Serial.print(ei_classifier_inferencing_categories[i]);
    }
    Serial.println();
}

/* =====
 * 6. setup() - step2 + [+ STEP 4-C] BLE 초기화 5단계
 * ===== */
void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 5000) ;

    Serial.println("# Vision Gauge v3 - USB + BLE");
    malloc_addblock((void*)0x30000000, 288 * 1024);

    if (!ei_camera_init()) { Serial.println("# FATAL: 카메라"); while(1); }

    /* [+ STEP 4-C 추가] BLE 초기화 5단계 (순서 바꾸면 동작 안 함)
     * 1) BLE.begin()           BLE 스택 기동
     * 2) setLocalName()        기기 이름 (스캔 시 표시)
     * 3) setAdvertisedService() 광고에 서비스 UUID 포함
     * 4) addCharacteristic + addService 특성을 서비스에 등록
     * 5) advertise()          광고 시작 (Central이 발견 가능)
     */
    if (!BLE.begin()) {
        Serial.println("# FATAL: BLE.begin failed");
        while(1);
    }
    BLE.setLocalName(BLE_DEVICE_NAME);
    BLE.setAdvertisedService(visionService);
    visionService.addCharacteristic(resultChar);
    visionService.addCharacteristic(labelChar); // [+ 라벨 특성] 등록
    BLE.addService(visionService);

    // 초기값 [0, 0, 0] - 첫 연결 시 Central이 읽으면 이 값을 받음
    uint8_t initVal[3] = {0, 0, 0};
    resultChar.writeValue(initVal, 3);

    // [+ 라벨 특성] 라벨 목록을 십표로 이어 문자열로 기록
    // 예: "bottle,marker,unknown"
    // 제어 노드(ESP32)가 연결 직후 이 값을 read해서 LABELS[]를 채움
    String labelStr = "";
    for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
        if (i) labelStr += ",";
        labelStr += ei_classifier_inferencing_categories[i];
    }
    labelChar.writeValue(labelStr.c_str());
    Serial.print("# 라벨 특성 기록: "); Serial.println(labelStr);

    BLE.advertise();

```

```

Serial.print("# BLE advertising as ");
Serial.print(BLE_DEVICE_NAME);
Serial.println("");

// 라벨 목록 송신 (sendLabels로 분리 - loop에서 주기 재전송)
sendLabels();
Serial.println("# ready");
}

/* =====
 * 7. loop() - step2 + [+ STEP 4-C] BLE.poll() + BLE notify
 * ===== */
void loop() {
  /* [+ STEP 4-C 추가] BLE 이벤트 처리 - 매 루프 반드시 호출!
   * 누락 시 증상:
   *   - 연결은 되지만 1~2초 후 끊어짐 (Central은 PHY layer만 보고 alive 판단)
   *   - 학생 차시 BLE 문제의 80%가 이 한 줄 누락
   * 위치는 loop() 첫 줄이 권장 - 추론 사이에 끼면 추론 시간이 길어짐
   */
  BLE.poll();

  static uint32_t lastTick = 0;
  if (millis() - lastTick < LOOP_INTERVAL_MS) return;
  lastTick = millis();

  // 라벨 목록 주기적 재전송 (5초마다) - monitor.py 나중 연결 대비
  static uint32_t lastLbl = 0;
  if (Serial && millis() - lastLbl > 5000) {
    lastLbl = millis();
    sendLabels();
  }

  uint32_t t0 = millis();
  if (!ei_camera_capture(EI_CLASSIFIER_INPUT_WIDTH, EI_CLASSIFIER_INPUT_HEIGHT)) {
    ea_free(ei_camera_capture_out); return;
  }

  // 추론 (step1/step2와 동일)
  ei::signal_t signal;
  signal.total_length = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT;
  signal.get_data      = &ei_camera_get_data;
  ei_impulse_result_t result = {0};
  if (run_classifier(&signal, &result, false) != EI_IMPULSE_OK) {
    ea_free(ei_camera_capture_out); return;
  }

  // argmax
  float maxConf = 0; int maxIdx = -1;
  for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {
    if (result.classification[i].value > maxConf) {
      maxConf = result.classification[i].value; maxIdx = i;
    }
  }
  uint32_t totalMs = millis() - t0;

  /* USB 송신 (step2와 동일) - BLE와 독립적으로 동작 */
  if (Serial) {
    Serial.print("RES:");
    Serial.print(maxIdx); Serial.print(",");
    Serial.print(maxConf, 3); Serial.print(",");
    Serial.println(totalMs);

    if (loopCount % IMG_EVERY_N == 0) {
      Serial.print("IMG:");
      size_t pixBytes = EI_CLASSIFIER_INPUT_WIDTH * EI_CLASSIFIER_INPUT_HEIGHT * 3;
      serialPrintBase64(ei_camera_capture_out, pixBytes);
      Serial.println();
    }
  }
}

```

```

    }
}
loopCount++;

/* [+ STEP 4-C / 절충안] BLE 실시간 송신
 * STABLE 조건 없이 매 추론 결과를 200ms마다 최신값으로 송신.
 *   - 즉각 반응 (확신도 낮아도 보임)
 *   - 200ms 쓰로틀로 OLED 깜빡임 억제 (매 추론마다 보내면 너무 잦음)
 * monitor.py는 이 송신을 BLE 인디케이터에 반영. */
static uint32_t lastBleMs = 0;
if (millis() - lastBleMs >= BLE_TX_INTERVAL_MS) {
    lastBleMs = millis();
    seqNum++;

    // 3바이트 [class_id, conf*100, seq] - 가장 확신도 높은 클래스 그대로
    uint8_t bleBuf[3] = {
        (uint8_t)maxIdx,
        (uint8_t)(maxConf * 100), // 0.0~1.0 -> 0~100 정수
        seqNum
    };
    resultChar.writeValue(bleBuf, 3);

    // 사람용 로그 (monitor.py가 BLE 송신 인디케이터 트리거로 사용)
    Serial.print("# TX: ");
    Serial.print(ei_classifier_inferencing_categories[maxIdx]);
    Serial.print(" "); Serial.print(maxConf, 2);
    if (BLE.connected()) {
        Serial.println(" (BLE central에 notify 전송)");
    } else {
        Serial.println(" (BLE 미연결 - 수신자 없음)");
    }
}

ea_free(ei_camera_capture_out);
}

#ifdef EI_CLASSIFIER_SENSOR || EI_CLASSIFIER_SENSOR != EI_CLASSIFIER_SENSOR_CAMERA
#error "이 펌웨어는 카메라 입력 모델 전용입니다"
#endif

```

4-C-3. 검증 — 폰 nRF Connect (제어 노드 만들기 전 단독 확인)

제어 노드를 만들기 전에 폰 BLE 디버거로 먼저 검증하면 문제를 절반으로 줄입니다. STEP 06에서 OLED가 안 켜졌을 때 "BLE가 안 가는지, OLED가 안 받는지" 모호한 상황을 피할 수 있습니다.

1. 학생 폰에 **nRF Connect for Mobile** 설치 (Nordic 제공, iOS·Android 모두 무료)
2. 비전 노드 부팅 → 시리얼에 "BLE advertising as 'VisionGauge'" 확인
3. nRF Connect → SCAN → "VisionGauge" CONNECT
4. Unknown Service의 **0x7e400002** 특성 → **구독(Notify)** 아이콘 탭
5. 카메라 앞에 학습한 물체 → Value 영역에 3바이트 hex 표시 변화 확인

3바이트 해석법

[0] 클래스 인덱스 (0~N-1, 알파벳순) · [1] 확신도 (0~100 정수, $\text{conf} \times 100$) · [2] 시퀀스 (0~255 순환). 예: 0x01 0x5A 0x03 = 두 번째 라벨, 확신도 90%, 세 번째 갱신. 시퀀스는 ESP32가 "같은 결과를 또 받았는지(no-op) / 새 갱신인지" 판단하는 데 씁니다.

송신 방식 — 실시간 vs 안정화 (설계 선택)

이 코드는 **200ms마다 최신 추론 결과를 그대로 송신**하는 실시간 방식입니다. 안정화 필터(STABLE: 확신도 0.70 이상 + 같은 답 3회 연속)를 통과한 결과만 보내는 방식도 있지만, 그 경우 **확신도가 임계값을 못 넘으면 아무것도 안 보내** OLED가 비어 보일 수 있습니다. 학생 물체·조명 조건에서 확신도가 낮게 나오는 경우가 많아, 교육용으로는 **실시간 방식이 "즉각 반응"을 보여주기에 유리**합니다. 대신 확신도가 낮을 땐 라벨이 자주 바뀌는데, 이를 200ms 쓰로틀로 완화하고 OLED는 받은 값을 부드럽게 표시합니다. **"즉각성(실시간) vs 안정성(필터)"의 트레이드오프**를 학생에게 설명하는 좋은 소재입니다 —

`BLE_TX_INTERVAL_MS` 값을 바꿔 체감 차이를 실험해볼 수도 있습니다.

4-C-4. 운영 트러블슈팅 (학생 차시 빈발)

증상별 진단

- ① **"BLE.begin failed" 시리얼 출력** — 보드 패키지 구버전. 보드 매니저 "Arduino Mbed OS Nicla Boards" 최신 업데이트.
- ② **nRF Connect에 "VisionGauge" 안 보임** — 폰 BLE 권한 미허용, 또는 이전 세션이 점유 중. 폰 블루투스 토글 + 앱 강제 종료 후 재시도.
- ③ **연결 직후 1~2초 만에 끊김 (가장 흔함)** — `BLE.poll()` 누락. step3 코드의 `loop()` 첫 줄에 있는지 확인. 학생 차시 80%의 BLE 문제 원인.
- ④ **연결은 됐지만 3바이트 미갱신** — 안정화 필터 미통과. 시리얼에 STABLE 줄이 안 뜨는 게 증거 → confidence 0.70 미만이거나 같은 답 3회 연속 안 됨. `CONF_THRESHOLD`를 0.60으로 낮추거나 학습 데이터 보강.
- ⑤ **추론 시간이 step2 대비 갑자기 늘어남** — BLE 작업이 추론 시간에 끼어드는 정상 현상. 5~10ms 정도면 정상. 30ms 이상 늘어나면 `BLE.poll()` 호출 위치 점검 (`loop()` 첫 줄 권장).
- ⑥ **USB monitor.py와 BLE 동시 사용 시 끊김** — Nicla의 RAM 한계. step3는 BLE 스택만 약 30KB 추가 사용. 시연 시 PC 안 쓰면 시리얼 모니터를 달아두는 게 안전.

왜 폰 검증을 권하나

제어 노드(ESP32+OLED) 한 번에 합쳐 디버깅하면 "안 되는데 어디서 막혔는지" 모호합니다. **비전 노드를 폰으로 확인 → ESP32+OLED 추가**의 2단계 분리가 학생 차시 디버깅 시간을 절반으로 줄입니다. 폰 nRF Connect는 사실상 "BLE 만능 멀티미터" — STEP 06으로 넘어가기 전 반드시 거치게 안내하세요.

모니터 UI 동작 원리 — monitor.py 들여다보기

담당: PC (Python + Streamlit) — 4-B에서 이미 실행됨. 본 STEP은 동작 원리와 확장 운영 팁.

WHAT — monitor.py 구조

백그라운드 스레드가 시리얼 줄을 읽어 머리말로 분기 — LBL은 라벨 동기화, RES는 게이지·이력 갱신, IMG는 base64 디코딩 → numpy 배열 → 영상 표시, "# STABLE"은 확정 박스 갱신. 메인 스레드는 200ms 주기로 화면을 갱신하다 10초마다 st.rerun.

WHY — 한 포트로 다 받는 설계

비전 노드가 머리말로 줄을 구분해 보내므로 PC는 포트 하나만 열어도 영상·결과·라벨이 동시에 들어옵니다. 시리얼 모니터 수준의 단순함과 풍부한 시각화의 균형.

5-1. 줄 단위 프로토콜 — 4종

줄	처리	오류 시 동작
LBL:<라벨1>,<라벨2>,...	쉼표 split → 라벨 리스트 보관. 예: LBL:bottle,marker,pen	비어 있으면 인덱스로 표시
RES:0,0.894,32	int·float·int 파싱 → 메인 화면 + 이력 큐	파싱 실패 시 조용히 버림
IMG:<base64>	b64decode → 12,288바이트 검증 → reshape(H,W,3)	크기 다르면 디코드 무시 (다음 프레임에서 복구)
# STABLE: <라벨> <conf> seq=N	토큰 파싱 → 확정 박스 갱신	파싱 실패 시 그대로 두기

5-2. 화면 구성

- **사이드바**: 포트 목록(자동 검색) + 연결/정지 + 상태 표시
- **왼쪽**: 모델 시점 64×64 영상을 5배 확대(픽셀이 보이는 것이 의도 — "AI는 이 정도 해상도로 본다"를 학생에게 시각적으로 전달)
- **오른쪽 위**: 실시간 인식 (st.metric + progress bar)
- **오른쪽 아래**: 안정화 확정 결과 (초록 박스, 가장 최근 STABLE)
- **하단**: 최근 20회 이력 표 (시각·클래스·확신도·추론ms)

5-3. 운영·확장 노트

- **10초 rerun 주기:** Streamlit의 위젯 점유·메모리 누수 방지 목적. 시연 중에는 자연스러운 깜빡임. 학생이 이상해하면 의도된 동작임을 설명.
- **색 채널 반전:** 영상이 푸르스름·붉으스름하면 펌웨어의 RGB565→RGB888 변환 매크로가 BGR로 풀어주는 경우. monitor.py에서 reshape 후 `[:, :, ::-1]` 추가.
- **로그 영속화:** 이력 큐를 SQLite로 적재하면 "오늘 가장 많이 인식된 클래스"·"시간대별 분포" 분석 가능 — 기존 MQTT/SCD30 프로젝트의 시리얼→SQLite 패턴 그대로 이식 가능.
- **다국어 라벨:** LBL의 영문 라벨을 한글로 매핑하는 사전을 두면, 학습 데이터는 영문(티 라벨 안정성 ↑) + 화면 표시는 한글로.
- **학생 미션 ②**의 코드 개선 미션에서 가장 손대기 쉬운 곳이 이 monitor.py입니다 — 게이지 디자인, 색 보정, 이력 분석. 펌웨어를 안 건드려도 시각적 변화가 큼니다.

학생 차시 운영 — 시리얼 포트 충돌

같은 COM 포트는 한 프로그램 점유. **Arduino IDE 시리얼 모니터 ↔ monitor.py ↔ EI Studio 데몬** 셋이 같은 자원을 다룹니다. 다음 순서가 안전:

펌웨어 업로드 (IDE 모니터 닫혀 있어야) → 시리얼 모니터로 5초 확인 → 닫기 → monitor.py 실행. 재업로드 시엔 monitor.py 사이드바의 ■ 정지 먼저.

STEP 06

제어 노드 — 무선 출력 (BLE 수신 → OLED 표시 + 심화: 서보)

담당: Arduino Nano ESP32 + Grove 실드 — BLE Central로 비전 노드 결과를 받아 OLED에 표시. 서보는 심화 부록.

WHAT — 이 단계가 하는 일

BLE Central로서 비전 노드를 찾아 연결하고 notify를 구독합니다. 3바이트가 도착하면 클래스 번호를 라벨로 바꿔 OLED에 글자로 표시. PC 없이 보드 두 개로 시연이 완성됩니다.

WHY — OLED를 기본으로 한 이유

서보는 외부 5V 전원, LM2596 강압 모듈, 공통 GND 합류 배선이 필요해 학생 차시에서 모든 모듈이 동시에 운영하기엔 부담이 큼니다. OLED는 Grove I2C 케이블 하나로 끝나고 결과를 직접적으로 보여줍니다. 서보는 STEP 6-5 심화 부록으로 분리 운영.

6-1. 배선 — Grove 케이블 한 줄

장치	연결	전압
Grove OLED 1.12" V2 (SH1107G, 128×128)	Grove 실드의 I2C 포트 에 케이블 1줄로 연결	3.3V (자동, 실드에서 공급)

실드 전원 스위치는 3V3 고정

Nano ESP32는 3.3V 로직 보드. 실드 VCC 스위치를 5V로 올리면 ① 5V 레일은 보드 뒷면 VUSB 점퍼 미납 상태에선 죽어있고, ② 스위치가 8포트 공통이라 이후 꽂는 모든 Grove 센서가 5V 구동되어 3.3V 핀에 5V 신호가 들어가는 손상 경로가 열립니다(Seeed 공식 경고). OLED는 3.3V로 정상 동작합니다.

왜 이 배선인가 — 한 줄 원리

Grove 시스템의 4핀 케이블에 전원(3.3V·GND)과 I2C 신호(SCL·SDA)가 모두 모여 있어, 학생이 핀을 잘못 꽂을 가능성이 없습니다. 실드의 I2C 포트 어디든 동일 결과.

6-2. 라이브러리

1. Arduino IDE 라이브러리 매니저 → **"U8g2" by oliver** 설치 (Grove OLED V2/V3 모두 지원하는 표준 라이브러리)
2. "ESP32 BLE Arduino"는 esp32 보드 패키지에 기본 포함 — 별도 설치 불필요

V1/V2/V3 칩 차이 주의

같은 1.12" 제품이라도 버전마다 컨트롤러가 다릅니다:

V1.0: SSD1327, 96×96, I2C 0x3E → [U8G2_SSD1327_SEEED_96X96_F_HW_I2C](#)

V2.0/V2.1: SH1107G, 128×128, I2C 0x3C → [U8G2_SH1107_SEEED_128X128_F_HW_I2C](#) ← 본 매뉴얼 기준

V3.0: SH1107, 128×128 (SPI/IIC 듀얼) → V2와 동일 생성자

화면이 안 나오는 원인의 80%가 버전-생성자 불일치입니다. 학생 차시 진입 전 보드 실크 표기 확인 + I2C 스캐너 스케치로 주소 검증을 권장.

6-3. 펌웨어

```
/* =====  
 * Vision Gauge - 제어 노드 (Nano ESP32 + Grove OLED 1.12" V2)  
 * =====  
 * BLE Central로서 비전 노드에 연결해 인식 결과를 OLED에 표시.  
 *  
 * 핵심 동작:  
 * 1) 비전 노드("VisionGauge") 탐색 -> 연결
```

```

* 2) 라벨 특성(0x...003)을 read -> LABELS[] 동적 구성
* (비전 노드에서 학습한 라벨이 자동으로 따라옴 - 코드 수정 불필요)
* 3) 결과 특성(0x...002) 구독 -> 3바이트 수신 시 OLED 갱신
* 4) 상단에 BLE 연결/수신 상태 인디케이터 표시
*
* OLED: Grove 1.12" V2 (SH1107G, 128x128, I2C)
* 라이브러리: U8g2 by oliver, ESP32 내장 BLE
* ===== */
#include <BLEDevice.h>
#include <Wire.h>
#include <U8g2lib.h>

// ----- Grove OLED 1.12" V2 (SH1107G, 128x128) -----
U8G2_SH1107_SEEED_128X128_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);

// ----- BLE UUID (비전 노드와 동일해야 함) -----
#define SERVICE_UUID "7e400001-b5a3-f393-e0a9-e50e24dcca9e"
#define CHAR_UUID "7e400002-b5a3-f393-e0a9-e50e24dcca9e" // 결과 (3바이트)
#define LABEL_UUID "7e400003-b5a3-f393-e0a9-e50e24dcca9e" // 라벨 문자열

// ----- 라벨 (BLE로 자동 수신 - 직접 입력 불필요) -----
#define MAX_LABELS 8
String LABELS[MAX_LABELS]; // 비전 노드에서 받은 라벨이 채워짐
int numLabels = 0;

// ----- 상태 변수 -----
volatile int curClass = -1; // 현재 표시 중인 클래스 인덱스
volatile int curConf = 0; // 확신도 (0~100)
volatile bool updated = false; // 새 데이터 도착 플래그
volatile uint32_t lastRxMs = 0; // 마지막 수신 시각 (수신 인디케이터용)
bool bleConnected = false;

BLEClient* pClient = nullptr;
BLERemoteCharacteristic* pResultChar = nullptr;

/* ----- 결과 특성 notify 콜백 -----
* 비전 노드가 200ms마다 푸시하는 3바이트 [class_id, conf, seq] 수신 */
static void notifyCB(BLERemoteCharacteristic* c,
                    uint8_t* data, size_t len, bool isNotify) {
    if (len < 3) return;
    curClass = data[0]; // 클래스 인덱스 (LABELS[] 미수신이어도 저장)
    curConf = data[1]; // 확신도 0~100 정수
    updated = true;
    lastRxMs = millis(); // 수신 시각 기록 -> 인디케이터 점멸
}

/* ----- 비전 노드 연결 + 라벨 수신 ----- */
bool connectVisionNode() {
    BLEScan* scan = BLEDevice::getScan();
    scan->setActiveScan(true);

    // esp32 2.0.x: start()가 값을 반환 (3.x면 BLEScanResults* + res->)
    BLEScanResults res = scan->start(5);
    for (int i = 0; i < res.getCount(); i++) {
        BLEAdvertisedDevice d = res.getDevice(i);
        if (d.haveServiceUUID() && d.isAdvertisingService(BLEUUID(SERVICE_UUID))) {
            pClient = BLEDevice::createClient();
            if (!pClient->connect(&d)) return false;

            BLERemoteService* svc = pClient->getService(SERVICE_UUID);
            if (!svc) return false;
        }
    }
}

```

```

// --- 라벨 특성 read -> LABELS[] 구성 ---
BLERemoteCharacteristic* lblChar = svc->getCharacteristic(LABEL_UUID);
if (lblChar && lblChar->canRead()) {
    // 비전 노드가 라벨을 기록하기 전에 읽힐 수 있어 최대 5회 재시도
    String csv = "";
    for (int retry = 0; retry < 5; retry++) {
        csv = String(lblChar->readValue().c_str());
        csv.trim();
        if (csv.length() > 0) break;    // 값을 받으면 종료
        delay(300);                    // 비었으면 잠시 후 재시도
    }
    Serial.print("라벨 특성 read: ["); Serial.print(csv); Serial.println("]");

    if (csv.length() > 0) {
        numLabels = 0;
        int start = 0;
        while (numLabels < MAX_LABELS) {
            int comma = csv.indexOf(',', start);
            if (comma < 0) {
                LABELS[numLabels++] = csv.substring(start);
                break;
            }
            LABELS[numLabels++] = csv.substring(start, comma);
            start = comma + 1;
        }
        Serial.print("라벨 "); Serial.print(numLabels); Serial.println("개 파싱 완료");
        for (int k = 0; k < numLabels; k++) {
            Serial.print("  ["); Serial.print(k); Serial.print("] ");
            Serial.println(LABELS[k]);
        }
    } else {
        Serial.println("경고: 라벨 특성이 비어 있음 - 비전 노드 step3 최신본인지 확인");
    }
} else {
    Serial.println("경고: 라벨 특성(0x...003)을 못 찾음 - UUID 확인");
}

// --- 결과 특성 구독 ---
pResultChar = svc->getCharacteristic(CHAR_UUID);
if (!pResultChar || !pResultChar->canNotify()) return false;
pResultChar->registerForNotify(notifyCB);
return true;
}
}
return false;
}

/* ----- 상단 상태 바 그리기 -----
 * 좌측: BLE 연결 아이콘 / 우측: 수신 점멸 표시 */
void drawStatusBar() {
    oled.setFont(u8g2_font_5x8_tr);

    // BLE 연결 상태 (좌상단)
    if (bleConnected) {
        oled.drawStr(2, 8, "BLE:ON");
        // 연결 아이콘 (채워진 원)
        oled.drawDisc(40, 5, 3);
    } else {
        oled.drawStr(2, 8, "BLE:--");
        oled.drawCircle(40, 5, 3); // 빈 원
    }
}

```

```

}

// 수신 인디케이터 (우상단) - 최근 600ms 안에 수신했으면 채움
bool rxActive = (millis() - lastRxMs) < 600;
oled.drawStr(86, 8, "RX");
if (rxActive) {
    oled.drawBox(104, 1, 8, 8);          // 채워진 사각 (수신 중)
} else {
    oled.drawFrame(104, 1, 8, 8);        // 빈 사각 (대기)
}

oled.drawLine(0, 12, 128);              // 구분선
}

/* ----- 메인 화면 그리기 ----- */
void drawScreen() {
    oled.clearBuffer();
    drawStatusBar();

    if (!bleConnected) {
        oled.setFont(u8g2_font_ncenB10_tr);
        oled.drawStr(8, 60, "connecting");
        oled.drawStr(8, 80, "...");
    } else if (curClass < 0) {
        oled.setFont(u8g2_font_ncenB10_tr);
        oled.drawStr(8, 60, "waiting");
        oled.drawStr(8, 80, "for data");
    } else {
        // --- 인식된 라벨 ---
        // LABELS[]가 비었거나 범위 밖이면 인덱스로 fallback (디버깅용)
        String lab;
        if (curClass >= 0 && curClass < numLabels && LABELS[curClass].length() > 0) {
            lab = LABELS[curClass];
        } else {
            lab = "C" + String(curClass);    // 라벨 미수신 시 "C0", "C1"... 로 표시
        }

        // 라벨 길이에 따라 폰트 자동 선택 (긴 라벨도 안 잘림)
        if (lab.length() <= 6)      oled.setFont(u8g2_font_ncenB18_tr);
        else if (lab.length() <= 9) oled.setFont(u8g2_font_ncenB14_tr);
        else                       oled.setFont(u8g2_font_ncenB10_tr);
        oled.drawStr(6, 50, lab.c_str());

        // --- 확신도 숫자 ---
        char buf[24];
        snprintf(buf, sizeof(buf), "conf %d%%", curConf);
        oled.setFont(u8g2_font_6x12_tr);
        oled.drawStr(6, 78, buf);

        // --- 확신도 bar 게이지 ---
        int barX = 6, barY = 88, barW = 116, barH = 16;
        oled.drawFrame(barX, barY, barW, barH);          // 외곽
        int fillW = (curConf * (barW - 4)) / 100;        // 채움 길이
        oled.drawBox(barX + 2, barY + 2, fillW, barH - 4); // 내부 채움

        // --- 눈금 (25/50/75%) ---
        for (int p = 25; p < 100; p += 25) {
            int tx = barX + 2 + (p * (barW - 4)) / 100;
            oled.drawVLine(tx, barY + barH + 2, 3);
        }
        oled.setFont(u8g2_font_4x6_tr);
    }
}

```

```

oled.drawStr(barX, barY + barH + 12, "0");
oled.drawStr(barX + barW - 16, barY + barH + 12, "100");
}

oled.sendBuffer();
}

void setup() {
  Serial.begin(115200);
  delay(500);           // 시리얼 안정화 대기
  Serial.println();
  Serial.println("=== 제어 노드 부팅 시작 ===");

  Wire.begin();
  Serial.println("1) I2C 초기화 완료");

  oled.begin();
  Serial.println("2) OLED 초기화 완료");
  drawScreen();         // "connecting..." 표시

  BLEDevice::init("");
  Serial.println("3) BLE 초기화 완료 - 비전 노드 탐색 시작");

  int tries = 0;
  while (!connectVisionNode()) {
    tries++;
    Serial.print("비전 노드 재탐색... ("); Serial.print(tries); Serial.println("회)");
    delay(1000);
  }
  bleConnected = true;
  Serial.println("4) 비전 노드 연결 성공!");
  drawScreen();         // "waiting for data" 표시
}

void loop() {
  // 새 데이터가 왔거나, 수신 인디케이터 점멸을 위해 주기적으로 다시 그림
  static uint32_t lastDraw = 0;
  if (updated || (millis() - lastDraw > 200)) {
    updated = false;
    lastDraw = millis();
    drawScreen();
  }
  delay(20);
}

```

U8g2의 그리기 패턴

`clearBuffer()` → `drawXxx()` → `sendBuffer()` 의 더블 버퍼 방식. 매번 화면 전체를 메모리에 그린 후 한 번에 출력하므로 깜빡임이 없습니다. 이 코드는 새 데이터가 오거나 200ms마다 갱신 — 수신 인디케이터 점멸을 위해 주기적 재그리기가 필요.

제어 노드 코드에는 `LABELS[]` 를 하드코딩하지 않습니다. 대신 **BLE 서비스에 라벨 특성(0x...003)**을 추가해, 비전 노드가 부팅 시 `"label1, label2, unknown"` 형태로 기록하고, 제어 노드가 연결 직후 `readValue()` 로 읽어 `LABELS[]` 를 동적 구성합니다. 결과 특성(3바이트, 자주 갱신)과 라벨 특성(문자열, 1회 기록)을 분리한 이유는 — 자주 바뀌는 작은 데이터와 거의 안 바뀌는 큰 데이터를 같은 특성에 섞으면 비효율적이기 때문. **"데이터의 변경 빈도에 따라 채널을 나눈다"**는 통신 설계 원칙의 좋은 예시로 학생에게 설명할 수 있습니다.

"Label 1, Label 2"로 표시되는 문제 — 진단

OLED에 실제 라벨 대신 인덱스성 텍스트가 뜨면: ① 비전 노드가 **라벨 특성을 보내는 step3 최신 버전**인지 확인 (구버전은 라벨 특성이 없음), ② 제어 노드 시리얼 모니터에 연결 직후 "라벨 수신: bottle,marker,..." 줄이 출력되는지 확인. 안 뜨면 라벨 특성 read 실패 — UUID(0x...003) 일치 여부 점검. ③ `MAX_LABELS` (기본 8)를 학습 클래스 수가 초과하지 않는지 확인.

6-4. 동작 확인

1. 비전 노드: STEP 4-C BLE 송신 추가본 업로드 후 동작 중 (LBL 줄에 라벨 출력 확인)
2. 제어 노드: 위 코드 업로드 → 부팅 → OLED에 "Vision Gauge / connecting..." 표시
3. 5~10초 안에 "waiting for data"로 전환되면 BLE 연결 성공. 시리얼 모니터에 "재탐색"이 5번 이상 반복되면 UUID·기기명 불일치 의심.
4. 비전 노드 앞에 학습한 물체(안경 등)를 들이대면 OLED에 라벨·확신도·바 그래프 표시
5. 물체 교체 시 안정화 지연(0.6초) 후 OLED 표시 변경 — 즉시 안 바뀌어도 정상

6-5. 심화 부록 — 서보 액추에이터 추가

OLED 글자 출력에 더해 물리적 게이지 바늘로 결과를 표시합니다. 외부 전원이 필요해 학생 차시 공통이 아닌 **도전 모듈용 확장 과제**로 운영.

추가 준비물

- SG90 서보 모터 1개
- LM2596 강압 모듈 (9~12V → 5.0V 가변 출력, FND 전압 표시형 권장)
- 9~12V DC 어댑터 1개 (보드 USB 전원과는 별도)
- Grove-to-male/female 점퍼, 또는 미니 브레드보드 + WAGO 커넥터

배선 표

연결	경로	비고
서보 신호 (주황)	실드 D6 → Grove-to-male 노랑 핀 → 서보 주황	흰색 핀은 NC
서보 전원 (빨강)	LM2596 OUT+ (5.0V) → 서보 빨강	모듈 입력 IN은 9~12V 어댑터
GND 공통	실드 검정 핀 + LM2596 OUT- + 서보 갈색 → 한 점 합류	미니 브레드보드/WAGO
Grove 빨강 핀 (3.3V)	아무 데도 연결 안 함	절연 테이프 마감

LM2596 설정 — "무부하 5.0V 확인 → 연결" 철칙

- ① 서보 미연결 상태에서 IN+/IN-에 9~12V 어댑터 연결 — 강압형이라 입력은 출력+1.5V 이상.
 - ② 버튼으로 FND 표시를 OUT 모드로 전환.
 - ③ 파란 트리머(W103) 일자 나사를 돌려 **5.0V** 표시까지 — 멀티턴(약 25회전)이라 초반 무반응 정상, 끝단 '딸깍'은 범위 한계 신호.
 - ④ 5.0V 확인 후에야 OUT+→서보 빨강, OUT-→GND 합류점 연결.
- 가변 모듈은 이전 사용자의 설정이 남아 있을 수 있으니 **전원 투입 시마다 FND 숫자 확인** 습관화.

왜 분리 급전인가

SG90 서보의 기동 전류는 100~500mA에 달해 ESP32의 3.3V 레귤레이터로는 감당 못 함 → 보드 brown-out 리셋 반복. 그리고 PWM 신호의 기준은 공통 GND — 외부 전원·서보·보드 GND를 한 점에서 합류시켜야 신호가 정확합니다. GND 안 합치면 서보가 부들거리거나 멧대로 회전.

코드 추가 (6-3에 덧붙임)

```
// ===== 6-3 코드에 다음을 추가 =====
#include <ESP32Servo.h>           // 라이브러리 매니저: "ESP32Servo"

Servo gauge;
const int ANGLES[] = { 0, 90, 180 }; // glasses=0°, keyboard=90°, unknown=180°
float curAngle = 180, targetAngle = 180;

// notifyCB() 안에 한 줄 추가:
targetAngle = ANGLES[cls];

// setup() 안에 추가:
gauge.attach(6);                 // Grove D6
gauge.write(180);                // 시작은 unknown 위치
```



```
// loop() 안에 추가 — easing으로 부드럽게:
static unsigned long t = 0;
if (millis() - t > 15) {
  t = millis();
  curAngle += (targetAngle - curAngle) * 0.15;
  gauge.write((int)(curAngle + 0.5));
}
```

다이얼 공작

1. A4에 반원(반지름 8~10cm) → 0°/90°/180° 위치에 라벨 표기 (3클래스면 양 끝과 중앙)
2. 먼저 `gauge.write(0)` / `gauge.write(180)` 으로 서보의 0°·180° 방향 확인 후 라벨 부착
3. 서보를 다이얼판 뒤에 양면테이프로 고정, 서보 혼에 빨대/하드보드 바늘 접착
4. 바늘은 서보 90° 상태에서 정중앙을 가리키도록 끼움 — 기계적 영점

연출 아이디어

unknown(180° = 우측 끝) 자리에 "?" 대신 "찾는 중..." 적기. confidence 값을 easing 계수에 매핑해 확신할수록 빠르게 도달하게 — 학생 미션 ②의 확장 주제로 좋습니다.

STEP 07

독립 시연용 웹 출력 (옵션)

PC 없이 시연할 때만 추가 — 제어 노드에 웹서버를 얹어 폰으로 게이지 확인 (영상은 코어측 모니터 전용)

옵션 ① SoftAP 독립 웹서버 ★ 권장

ESP32가 스스로 Wi-Fi 핫스팟이 됩니다. 폰 Wi-Fi에서 "VisionGauge" 접속 → `192.168.4.1`

```
// ----- [옵션①] STEP 06 코드에 추가: SoftAP + 웹서버 -----
#include <WiFi.h>
#include <WebServer.h>

const char* AP_SSID = "VisionGauge"; // 보드가 직접 만드는 Wi-Fi 이름
const char* AP_PASS = "12345678"; // 8자 이상 필수 (WPA2 규격)
WebServer server(80);

void handleStatus() {
  char buf[96];
  snprintf(buf, sizeof(buf),
    "{\"label\":\"%s\",\"cls\":%d,\"conf\":%d,\"angle\":%d}",
    LABELS[curClass], curClass, curConf, ANGLES[curClass]);
  server.send(200, "application/json", buf);
}
```

```
// setup()에 추가:
// WiFi.softAP(AP_SSID, AP_PASS);    // 보드 스스로 공유기가 됨
// Serial.println(WiFi.softAPIP());    // 항상 192.168.4.1
// server.on("/status", handleStatus);
// server.on("/", [](){ server.send(200, "text/html", INDEX_HTML); });
// server.begin();
// loop()에 추가: server.handleClient();
```

웹페이지(INDEX_HTML)는 스케치 상단에 문자열 상수로 넣습니다 — SVG 게이지가 0.4초마다 `/status`를 폴링해 물리 바늘과 동기화됩니다.

```
// ----- [옵션①/② 공용] INDEX_HTML: 스케치 상단에 추가 -----
const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html><html lang="ko"><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>비전 게이지</title>
<style>
body{font-family:sans-serif;background:#10211d;color:#e8f3ee;
    display:flex;flex-direction:column;align-items:center;padding-top:40px}
h1{font-size:20px;font-weight:600;letter-spacing:.05em}
#label{font-size:42px;font-weight:700;margin:10px 0 2px;color:#ffb454}
#conf{color:#7fb8a4;font-size:15px;margin-bottom:8px}
#needle{transition:transform .6s cubic-bezier(.2,1.4,.4,1);
    transform-origin:140px 150px}
</style>
<h1>VISION GAUGE</h1>
<svg width="280" height="170" viewBox="0 0 280 170">
  <path d="M30 150 A110 110 0 0 1 250 150" fill="none"
    stroke="#2e4b43" stroke-width="14" stroke-linecap="round"/>
  <g font-size="12" fill="#9fcabb" text-anchor="middle">
    <text x="26" y="168">병</text><text x="62" y="78">컵</text>
    <text x="140" y="32">가위</text><text x="218" y="78">펜</text>
    <text x="254" y="168">?</text>
  </g>
  <line id="needle" x1="140" y1="150" x2="140" y2="55"
    stroke="#ffb454" stroke-width="4" stroke-linecap="round"/>
  <circle cx="140" cy="150" r="8" fill="#e8f3ee"/>
</svg>
<div id="label">---</div><div id="conf"></div>
<script>
async function tick(){
  try{
    const r = await fetch('/status'); const d = await r.json();
    document.getElementById('needle').style.transform =
      `rotate(${d.angle - 90}deg)`;    // 서보 0~180° → SVG -90~+90°
    document.getElementById('label').textContent = d.label;
    document.getElementById('conf').textContent = `확신도 ${d.conf}%`;
  }catch(e){}
}
setInterval(tick, 400); tick();
</script>
)rawliteral";
```

SoftAP 운용 메모

① 접속한 폰은 그동안 인터넷이 끊깁니다 — "인터넷 없음" 경고가 떠도 '연결 유지' 선택. ② 동시 접속 ~4대 권장. ③ 대역폭은 낮지만 게이지 트래픽(초당 ~150바이트)에는 수만 배 여유. ④ 어느 장소로 옮겨도 재컴파일 불필요. ⑤ 이 옵션을 켜면 ESP32가 BLE+Wi-Fi 이중 무선이 되지만, "변화 시 3바이트 + 0.4초 폴링" 설계라 실용상 문제없습니다.

옵션 ② 공유기 경유 (Station) — 옵션①에서 세 줄 교체

```
// ----- [옵션②] WiFi.softAP(...) 부분을 아래로 교체 -----
const char* WIFI_SSID = "교실공유기"; // 환경마다 수정 + 재컴파일 필요
const char* WIFI_PASS = "비밀번호";

WiFi.begin(WIFI_SSID, WIFI_PASS);
while (WiFi.status() != WL_CONNECTED) delay(300);
Serial.println(WiFi.localIP()); // 이 IP를 시청자들에게 공지
```

옵션② 사전 확인 — AP isolation

학교·공공 Wi-Fi는 기기 간 통신을 차단하는 AP isolation이 켜진 경우가 많습니다 (이전 MQTT 프로젝트에서 겪었던 바로 그 문제). 이 경우 폰에서 ESP32의 IP로 접속이 안 됩니다. 시연 전 반드시 같은 망의 폰 → ESP32 접속 테스트를 하고, 차단돼 있으면 옵션①로 전환하세요.

STEP 08

게이지 다이얼 제작 (응용 예제)

담당: 손과 가위 — 디지털 결과를 물리 세계로 옮기는 공작 단계

WHAT — 이 단계가 하는 일

반원형 다이얼판에 물체 이름 레이블을 각도대로 배치해 부착하고, 서보 혼에 바늘을 달아 다이얼 중심에 고정합니다.

WHY — 왜 이렇게 하는가

화면 속 결과를 물리 바늘이 가리키는 순간, 추상적인 "추론"이 손에 잡히는 사건이 됩니다. 이 아날로그 변환이 데모의 재미 절반을 담당합니다.

- A4에 반원(반지름 8~10cm)을 그리고 `ANGLES[]` 와 동일한 각도에 레이블 표기. 5클래스면 0°/45°/90°/135°/180°.
- 방향 주의:** 테스트 스케치로 `gauge.write(0)` 을 실행해 서보 0°가 다이얼 어느 쪽인지 먼저 확인 후 레이블 부착.
- 서보를 다이얼판 뒤에 고정하고 서보 혼에 빨대/하드보드 바늘 부착.

4. 바늘은 서보 90° 상태에서 정중앙을 가리키도록 끼움 — 기계적 영점.

연출 아이디어

unknown(180°) 위치에 "?" 대신 "생각 중..."이라고 적으면 자연스러운 스토리가 됩니다. 확신도를 easing 계수에 매핑해 확신할수록 바늘이 빠르게 척 도달하게 하는 변형도 학생 과제로 좋습니다.

통합 테스트 절

STEP 09

차

아래 순서대로 하나씩 확인합니다. **한 번에 전부 켜고 "왜 안 되지"를 찾는 것이 최악의 디버깅입니다** — 각 단계는 앞 단계가 통과한 뒤에만 의미가 있습니다.

- ☐ ① **모델 단독** — EI Model testing에서 테스트 정확도 85%+ 확인
- ☐ ② **비전 노드 4-A 단독 추론** — vision_node_step1_test.ino 업로드 후 시리얼 모니터에서 RES/STABLE 줄 확인 (안정화 필터 동작 검증)
- ☐ ③ **비전 노드 4-B USB 스트리밍** — vision_node_step2_usb.ino + monitor.py 실행 → 영상·게이지·이력 동작 확인
- ☐ ④ **BLE 송신 검증** — 비전 노드에 BLE 송신 코드 추가 후, 폰 nRF Connect 앱으로 "VisionGauge" 연결 → 캐릭터리스틱 구독 → 물체 교체 시 3바이트 갱신 확인 (학생 차시에선 시간 절약 위해 다음 단계 ⑤에서 통합 검증 가능)
- ☐ ⑤ **제어 노드 OLED 표시** — Nano ESP32 + Grove OLED V2 (SH1107G, 128×128) 연결, control_node_oled.ino 업로드. 부팅 시 "connecting..." → "waiting for data" → 물체 인식 시 라벨·확신도·바 그래프 표시
- ☐ ⑥ **라벨 정합성** — 학습한 각 클래스 물체를 차례로 비춰 OLED 라벨과 일치 확인 (어긋나면 LABELS[]를 TI 라벨 알파벳순으로 재정렬, 학생 차시 가장 흔한 실수)
- ☐ ⑦ **내구 테스트** — 10분간 물체를 바꿔가며 방치, BLE 끊김·OLED 멈춤·리셋 없는지 관찰

☐ ⑧ (심화) 서보 추가 — STEP 6-5 진행 모듈: 서보 미연결 상태 LM2596 무부하 5.0V 확인 → 서보 연결 → 각도 회전 확인 → 바늘 영점(90°에서 정중앙) 조정

STEP 10

트러블슈팅 & 확장 과제

적용	증상	원인	해결
공통	Nicla 업로드 실패 / 포트 안 보임	부트로더 진입 필요	리셋 버튼 빠르게 2회(녹색 LED 점멸) 후 업로드
공통	BLE 코드 컴파일 오류	보드별 BLE 라이브러리 혼용	Nicla=ArduinoBLE, Nano ESP32=BLEDevice.h — 호환 불가
공통	비전 노드가 연결 직후 끊김	BLE.poll() 누락	loop()와 추론 루프 사이사이에 BLE.poll() 호출
공통	모델 빌드/업로드 시 메모리 부족	모델이 Nicla RAM 초과	Deployment에서 Quantized(int8), 입력 64×64 유지
모니터	대시보드가 포트를 못 엮	IDE 시리얼 모니터와 포트 충돌	둘 중 하나만 — IDE 모니터 닫고 streamlit 실행
모니터	영상이 안 뜸 / 간헐적으로 깨짐	base64 줄 잘림·길이 불일치	길이 검증(len==W*H*2)이 깨진 줄을 걸러줌 — 지속되면 보드 리셋
모니터	영상 색이 이상함 (보라/초록 틸트)	RGB565 바이트 오더	monitor.py의 dtype을 '>u2' ↔ '<u2' 교체
모니터	영상 fps가 낮음 (~5fps)	추론+전송 직렬 — 정상 사양	고fps가 필요하면 프레임을 N회에 1번만 송신하는 식의 역설계가 아니라, 이 수치가 설계값임을 안내
심화/서보	서보 동작 시 보드 리셋	보드 5V 직결 (brown-out)	외부 5V 분리 급전 + 공통 GND (STEP 6-5)
심화/서보	서보가 부들거리거나 멋대로 회전	공통 GND 누락	외부 전원 GND ↔ 보드 GND 한 점 합류 확인
심화/전원	LM2596 트림러 무반응	멀티턴(약 25회전) 특성	같은 방향으로 계속 — 끝단 '딸깍'은 범위 한계
심화/전원	출력 전압이 5V로 안 떨어짐	입력-출력 차 부족	강압형은 입력이 출력+1.5V 이상이어야 (9~12V 어댑터)
심화/전원	서보 연결 직후 파손 위험	이전 사용자의 고전압 설정 잔존	"무부하 5.0V 확인 → 연결" 철칙 — 시연 전 마다 FND 확인

OLED	화면이 까만 채 안 켜짐	Grove 케이블 헐거움 / I2C 아닌 포트	실드의 I2C 라벨 포트에 재연결, 양쪽 단자 단단히 결합
OLED	화면 켜지지만 깨진 패턴	버전-생성자 불일치 (V1=SSD1327, V2/V3=SH1107)	I2C 스캐너로 주소 확인 (V1=0x3E, V2/V3=0x3C) 후 생성자 교체. V2는 <code>U8G2_SH1107_SEEED_128X128_F_HW_I2C</code>
제어/컴파일	cannot convert 'BLEScanResults' to 'BLEScanResults*'	esp32 보드 패키지 버전별 API 차이 (2.0.x는 값 반환, 3.x는 포인터 반환)	2.0.x: <code>BLEScanResults res = scan->start(5); + res.</code> / 3.x: <code>BLEScanResults* res + res-></code> (STEP 06 코드 주석 참조)
제어/경고	"BLEDevice.h 복수 라이브러리 발견" 경고	ESP32 내장 BLE와 ArduinoBLE가 같은 헤더명 보유	에러 아님 — ESP32 보드 선택 시 내장 BLE를 올바르게 사용. 무시하고 진행
OLED	BLE 연결됐는데 화면 안 바뀜	updated 플래그 미 갱신 / notifyCB 미 등록	시리얼 모니터로 notifyCB 호출 확인 — 안 찍히면 BLE 구독 실패
OLED	한글 라벨 깨짐	U8g2 기본 폰트는 영문만	라벨을 영문 유지 또는 <code>u8g2_font_unifont_t_korean1</code> 같은 한글 폰트 사용 (플래시 ~150KB 추가)
OLED	(심화) OLED 갱신 시 서보 버벅임	매 루프 sendBuffer로 I2C 트래픽 증가	updated 플래그 시에만 sendBuffer 호출 (본 매뉴얼 구조 유지)
공통	OLED에 엉뚱한 라벨 표시	LABELS[] 순서 불일치	EI Studio 라벨 알파벳순과 코드 배열 순서 정확히 일치 확인 — 학생 차시 가장 흔한 실수
공통	Grove 센서 추가 후 보드 이상	실드 스위치 5V (3.3V 보드 손상 경로)	스위치 항상 3V3 — 서보 전원은 외부 모듈 전담
심화/서보	바늘이 엉뚱한 라벨 지시	라벨 순서 불일치 / 기계 영점	EI 라벨 알파벳순 확인, 90°에서 바늘 재장착
공통	인식이 흔들리거나 자주 바뀜	안정화 필터 약함	CONF_THRESHOLD ↑, STABLE_REQUIRED ↑ (펌웨어 상단 #define)

옵션 ①		폰이 모바일 데이터로 우회	"VisionGauge" 연결 확인, "인터넷 없음" 경고 시 '연결 유지', 모바일 데이터 잠시 끄기
옵션 ①②	웹 접속 시 BLE 끊김	무선 공존 한계 초과	송신 빈도 확인(변화시에만), 폴링 400ms 유지
옵션 ②	같은 망인데 IP 접속 불가	AP isolation (기기 간 통신 차단)	망 관리자 확인 또는 옵션①로 전환
공통	실물 인식 정확도가 낮음	학습-시연 환경 차이	실제 거리·배경·조명으로 데이터 재수집 (Nicla 카메라 직접 수집 권장)
공통	unknown만 계속 인식	물체가 프레임에서 작음	카메라-물체 거리 단축, 학습 사진 거리와 일치 — 코어측 모니터 영상으로 직접 확인 가능

확장 과제 (학생 도전용)

- **OLED 그래픽 게이지** — 글자 외에 U8g2의 drawBox/drawCircle/drawTriangle로 원형 게이지·아이콘 표시
- **탐색 모드** — unknown 5초 지속 시 OLED에 "찾는 중..." 애니메이션 (점 3개 순차 표시)
- **(심화) 서보 속도 = 확신도** — confidence를 easing 계수에 매핑, 확신할수록 바늘이 빠르게 도달
- **인식 통계** — monitor.py 이력을 SQLite에 적재, "오늘 가장 많이 인식된 물체" 대시보드
- **다중 액추에이터** — 클래스별 LED 색·부저 음 추가, Nicla 내장 RGB LED로 비전 노드 상태 표시
- **역방향 통신** — 모니터/웹 → BLE write → 비전 노드의 임계값 원격 조정
- **응용 전환** — 코어는 그대로 두고 분류 대상·액추에이터만 교체: 분리수거 분류 안내기, 재실 감지 도어락, 부품 검수 합불 표시기 등

비전 Edge AI 프로젝트 — 교사용 매뉴얼 · 2026-06 · 공주고등학교 SW·AI 융합교육

Nicla Vision (EI 분류) → [USB: IMG/RES → PC monitor.py] + [BLE 3-byte → Nano ESP32+Grove → OLED 1.12" V2 (SH1107G)] · 심화: 서보 액추에이터 (외부 5V 분리 급전)